Faculty of Sciences Department of Computer Sciences

Master thesis

Procedural Generation of Solvable Cooperative Levels for the Laser Learning Environment

Author:
Charels Hugo
hugo.charels@ulb.be

Supervisors:
Lenaerts Tom
Molinghen Yannick

Academic year 2025-2026

Contents

Chapter 1

Introduction

1.1 Context

Cooperative Multi-Agent Reinforcement Learning (MARL) studies settings in which several agents must learn behaviours whose value appears only at the team level. In such settings, the environment is not a neutral container: it determines which coordination patterns are possible, which ones are necessary, and how difficult they are to discover.

This dependence on environment structure is especially strong in sparse-reward cooperative tasks. When reward is issued only after the team objective has been achieved, a training level is useful only if it exposes a meaningful coordination challenge while remaining actually solvable. Unsolvable levels provide no valid signal, and levels that admit independent solutions fail to test the cooperative mechanism they are meant to study.

1.2 Motivation

In cooperative environments whose mechanics create explicit inter-agent dependencies, level design is particularly challenging. A useful training level should not merely appear cooperative — it should provably contain the intended coordination structure.

Procedural Content Generation (PCG) offers a way to scale level creation beyond what manual design allows, but only if the generated instances satisfy the properties that matter for training. Two properties are central. First, a level must be **solvable**. Second, success should **require cooperation** in the specific sense induced by the environment's mechanics. Without those guarantees, generation risks producing levels that are invalid, trivial, or misaligned with the training objective.

1.3 Research Questions and Scope

This thesis addresses the following question: how can we automatically generate levels for a cooperative MARL environment that are provably solvable and provably require the target cooperative interaction?

More precisely, the work is organised around five research questions:

- **RQ1:** How can we formally verify that a level is solvable by the agents?
- **RQ2:** How can we formally verify that solving a level genuinely requires cooperation between agents, rather than allowing independent solutions?
- **RQ3:** How can formal verification be embedded inside a procedural generator so that every accepted level comes with certified properties?
- **RQ4:** Can agents trained exclusively on procedurally generated levels transfer their behaviour to human-designed levels?
- **RQ5:** Can we control the cooperation structure of generated levels by targeting specific profiles such as asymmetric, mutual, chain, distributed, or fully coupled dependencies?

The thesis instantiates this framework on one concrete cooperative environment, focusing on a restricted but explicit subset of its mechanics. The formal model covers agent movement, inter-agent blocking interactions, and a bounded-horizon solvability criterion. Additional environment mechanics are outside the scope of the formal guarantees developed here. The broader methodology — coupling procedural generation with a formal verification oracle — is environment-agnostic: it applies to any setting where the target properties are expressible as decision problems.

1.4 Contributions

This thesis makes the following contributions:

- **A decision procedure for bounded-horizon solvability.** We provide a propositional encoding of the solvability decision problem over a bounded time horizon. The procedure either returns a certificate — a valid joint trajectory — or proves that no such trajectory exists within the chosen horizon (Section 4).
- **A formal cooperation detector.** We define a strict variant of the environment semantics in which agents cannot exploit same-agent blocking to bypass constraints imposed by others. A level requires cooperative interaction if and only if it is solvable under standard semantics and unsolvable under the strict variant (Section 5).
- **A solver-in-the-loop generation framework.** Building on the solver and cooperation detector, we implement six generators across two property families — solvable and cooperative. Each accepted level is certified by the solver to satisfy the advertised property (Section 6).
- **A cooperation profile taxonomy and profile-targeted generation.** We define a set of cooperation profiles — asymmetric, mutual, chain, distributed, fully coupled — that characterise the dependency structure of cooperative levels. Generators can target a specific profile, producing levels whose cooperation type is controlled, not merely certified (Section 5, Section 6).
- **An empirical evaluation.** We compare two alternative propositional encodings of the agent-uniqueness constraint, measuring their effect on formula size and solver runtime. We also measure generator acceptance rates and cooperation profile distributions across grid sizes (Section 7.1, Section 7).

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 positions the work relative to cooperative MARL benchmarks, procedural content generation, and compilation-based planning literature.

Chapter 3 fixes the technical setting: the LLE mechanics, the SAT background, and the formal model of bounded-horizon solvability and the cooperation requirement.

The three contribution chapters then develop the original technical content of the thesis in logical order. Chapter 4 introduces the SAT-based solver: a propositional encoding of bounded-horizon LLE solvability, with correctness proofs and a complexity-theoretic positioning. Chapter 5 builds on that encoding to define the cooperation detector based on a strict counterfactual semantics, and extends it with a profile analyzer that classifies the kind of cooperation a level exhibits. Chapter 6 then uses both decision procedures as acceptance oracles inside a family of procedural generators that produce levels certified to satisfy the advertised properties.

Chapter 7 reports the empirical evaluation: the SAT encoding comparison, generator rejection rates, the cooperation profile distribution, and the transfer experiment to human-designed levels. Chapter 8 concludes with a summary and directions for future work.

Chapter 2

Related Work

2.1 Cooperative MARL and Coordination-Critical Benchmarks

This thesis is motivated by a benchmark-design question within cooperative Multi-Agent Reinforcement Learning (MARL): how can one generate training instances whose coordination structure is both non-trivial and formally controlled?

In the fully cooperative setting, all agents optimise a shared return, so the quality of the environment matters as much as the quality of the learning algorithm. If the environment admits only trivial solutions, it does not meaningfully test coordination. If it contains unsolvable instances, it provides no useful training signal at all. For the present thesis, the central issue is therefore not MARL in general, but the design of instances in which inter-agent dependence is both structurally present and formally decidable.

The general framework for sequential multi-agent decision-making originates in stochastic games [1], generalised to the Markov game model that has become the standard formalism for MARL [2]. The single-agent reinforcement-learning machinery on which cooperative MARL builds is covered comprehensively in [3]. In cooperative MARL specifically, value-decomposition methods such as VDN [4] and QMIX [5] factor a shared team value into per-agent components to enable decentralised execution; closely related approaches include the centralized-critic actor-critic of MADDPG [6], the counterfactual-baseline policy gradient COMA [7], and MAVEN [8], which extends QMIX with latent-variable exploration to overcome its monotonicity limitations on coordination tasks. These methods perform well when individual contributions are roughly additive, but they struggle precisely on the coordination-critical, low-reward bottlenecks that LLE is designed to expose [9].

2.2 The Laser Learning Environment

The Laser Learning Environment (LLE) was introduced precisely to study coordination-critical multi-agent tasks [9]. The paper identifies three properties that make the benchmark difficult for value-based MARL methods: **perfect coordination**, **interdependence**, and **zero-incentive dynamics**. Together, these properties create bottlenecks in which one agent must perform a locally unrewarded action that enables another agent to progress.

This benchmark framing is directly relevant to the present work. The thesis does not attempt to improve MARL training algorithms on LLE. Instead, it addresses an upstream question left open by the

benchmark paper: how can we generate LLE levels that are guaranteed to be solvable and that contain the beam-blocking dependency on which the benchmark relies?

The LLE paper therefore plays two roles in this thesis. First, it justifies why LLE is an interesting target domain. Second, it provides the conceptual vocabulary used here to discuss cooperation: the key object is not generic teamwork, but a concrete interdependence mechanism created by coloured lasers and same-colour blocking.

2.3 Procedural Generation Under Structural Constraints

Procedural Content Generation (PCG) is useful in reinforcement-learning settings because it can replace a small fixed benchmark set with a larger and more diverse stream of instances [10]. Within PCG, search-based methods — surveyed by J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne [11] — phrase content creation as an optimisation problem over a content space, which is conceptually close to the solver-driven acceptance loop adopted in this thesis. The closest precedent for the present declarative-constraint approach is the Answer Set Programming generator of A. M. Smith and M. Mateas [12], which uses an ASP solver as the acceptance oracle for game content. A complementary line of work surveyed under the PCGML banner [13] uses machine-learned generators trained on existing levels, but typically provides no formal guarantee on the produced output. The difficulty for the present problem is not merely to produce varied levels, but to produce levels that satisfy logically defined properties.

This distinction matters. A constructive or search-based generator may bias generation toward interesting layouts, but without a verifier it cannot certify that a sampled level is solvable or that success genuinely depends on cooperation. For that reason, the present thesis adopts a constraint-aware view of PCG: generation is coupled to a formal decision procedure, and the solver acts as an acceptance oracle rather than as a post-hoc descriptive tool.

2.4 SAT-based Planning

A second compilation lineage directly relevant to this thesis is **SAT-based planning**. H. Kautz and B. Selman [14] introduced SATPLAN, encoding bounded-horizon STRIPS planning instances as propositional formulas; subsequent work refined both the encodings and the search strategies [15]. The treatment by J. Rintanen, K. Heljanko, and I. Niemelä [16] covers parallel-plan encodings and modern algorithmic refinements that drove SAT-based planners to competitive performance with dedicated planners. The bounded-horizon LLE encoding developed in this thesis sits in the same conceptual family: a state-transition decision problem reduced to SAT, with the encoding choices materially affecting solver performance.

2.5 Compilation-Based Multi-Agent Path Finding

The closest methodological precedent is not PCG for MARL, but compilation-based Multi-Agent Path Finding (MAPF). In standard MAPF, agents move on a discrete graph from start vertices to goal vertices while avoiding collisions. The computational difficulty comes from the interaction between multiple agents and the optimality criterion imposed on the solution.

The survey by P. Surynek [17] shows that MAPF has become a major testbed for compilation-based solving. Instead of searching directly in the original state space, one reduces a MAPF instance to a target formalism such as CSP, SAT, or MILP, then relies on the target solver to handle the combinatorial burden. MAPF research is not exclusively compilation-based; the dominant search-based alternative, Conflict-Based Search [18], achieves optimal solutions through a two-level constraint-satisfaction tree without reducing to a target formalism. The compilation survey is especially relevant here for two reasons.

First, it demonstrates that SAT-based reductions are a mature and credible way to solve structured multi-agent planning problems. Second, it makes clear that compilation is not a black-box slogan: modeling choices, encoding size, and the interaction between the source problem and the target solver all matter materially for performance.

The present thesis inherits this compilation perspective. Bounded-horizon LLE solvability is treated as a decision problem and is reduced to SAT. The difference is that LLE is not standard MAPF: the environment contains colour-dependent laser semantics and the property of interest is not path optimality, but solvability and cooperation under the benchmark mechanics.

2.6 SAT-Based MAPF Encoding Design

Beyond the general survey, the MAPF literature also provides concrete lessons about SAT encoding design. The paper by M. Frommknecht and P. Surynek [19] studies SAT-based MAPF solving under the makespan objective using an MDD-SAT formulation and compares different solver-facing choices, including eager versus lazy encodings and the use of informative initial assignments.

That paper is relevant to the present thesis not because it solves the same problem, but because it shows that SAT-based multi-agent solving is sensitive to representation details. Performance is not determined only by the underlying decision problem; it also depends on how the problem is encoded and on how the resulting CNF interacts with the chosen SAT solver. This is directly aligned with the experimental part of the current thesis, where two alternative uniqueness encodings are compared empirically.

At the same time, the distance between the two settings should be stated explicitly. Standard MAPF encodings reason about graph motion and collisions. The current thesis must additionally encode time-dependent laser propagation, same-colour immunity, and a strict counterfactual semantics used to define cooperation. The MAPF literature therefore supplies a methodological template, not a drop-in solution.

2.7 Positioning of the Thesis

The literature leaves a clear opening for the present work.

- The LLE paper [9] establishes the benchmark and explains why its coordination bottlenecks are difficult for MARL algorithms, but it does not provide a formal generator for certified cooperative levels.
- The MAPF compilation survey [17] shows that SAT is an effective backend for multi-agent planning problems, but it addresses standard MAPF rather than LLE-specific laser dynamics.

- The MAPF SAT-engineering paper [19] shows that encoding design affects solver performance in practice, but it remains within the standard MAPF framework and does not address cooperation as a semantic property of the instance.

This thesis sits at the intersection of those lines of work. It transfers the compilation-based SAT mindset from MAPF into the LLE setting, formalises bounded-horizon solvability for an LLE-specific model, and introduces a strict-semantics counterfactual that turns a benchmark-level intuition about cooperation into a decidable property inside procedural generation.

Chapter 3

Background and Formalization

3.1 The Laser Learning Environment

The Laser Learning Environment (LLE) [9] is a grid-based cooperative multi-agent benchmark designed to study coordination-critical tasks. A level consists of a rectangular grid containing walls, coloured laser sources, agent start positions, and exit tiles. Each agent is assigned a colour and must reach its designated exit tile.

The central mechanic is the laser beam. Each source emits a directional beam that propagates cell by cell until it reaches a wall or the grid boundary. An agent whose colour matches the beam is *immune* to it: it may occupy a cell the beam traverses without being harmed, but its physical presence truncates the beam at that cell. Agents of other colours cannot occupy a cell crossed by an active beam.

This truncation is the source of the cooperative structure studied in this thesis. By occupying a position along its own beam, an agent shortens the beam and opens cells beyond it for teammates who are not immune to that colour. Crucially, this action is locally unrewarded: the helper agent receives no direct benefit from blocking its own beam. The reward is issued only when all agents simultaneously occupy their exits, which requires the full team to coordinate successfully.

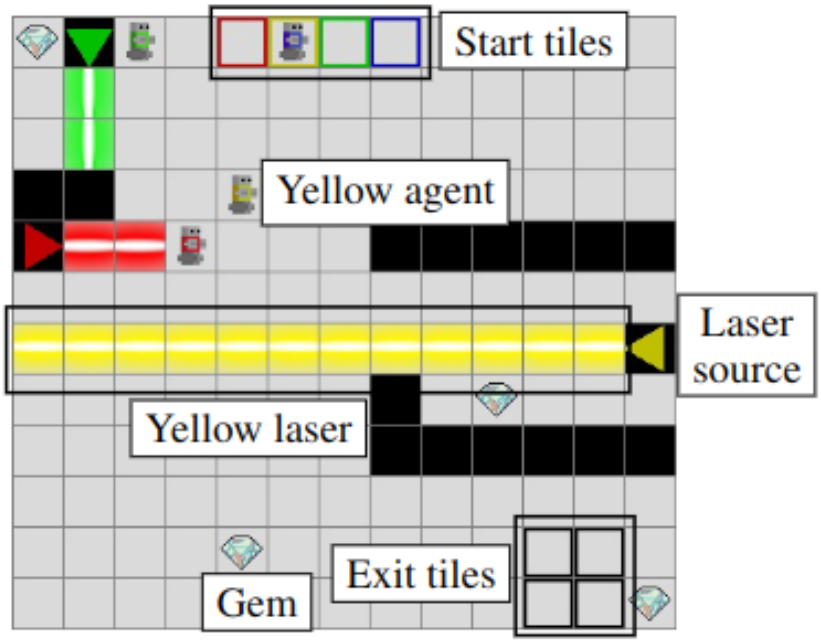

Figure 1: An annotated LLE level. Coloured agents must reach their matching exit tiles. Laser beams block movement for non-immune agents; an immune agent can truncate its own beam by occupying a cell along its path, opening a route for a teammate.

A complete description of the benchmark, its difficulty properties, and its relation to existing MARL methods was given in Chapter 2. The present chapter focuses on the formal model used to reason about solvability and cooperation within a bounded time horizon.

3.2 Boolean Satisfiability

Boolean Satisfiability (SAT) is the problem of deciding whether a propositional formula has a satisfying assignment — a mapping from Boolean variables to true or false that makes the formula evaluate to true. It is the canonical NP-complete decision problem [20].

In practice, SAT solvers operate on formulas in *Conjunctive Normal Form* (CNF). A CNF formula is a conjunction of *clauses*, where each clause is a disjunction of *literals* and a literal is either a variable x or its negation $\neg x$. For example:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3)$$

is a CNF formula over three variables. It is satisfied by setting $x_1 = \text{true}$, $x_2 = \text{false}$, $x_3 = \text{true}$, among other assignments.

Modern SAT solvers based on the DPLL procedure and Conflict-Driven Clause Learning (CDCL) can handle industrial instances with millions of variables and clauses [21]. Given a CNF formula, a solver either:

- returns a *satisfying assignment* that witnesses the formula is satisfiable (SAT), or
- certifies that *no* satisfying assignment exists (UNSAT).

The UNSAT certificate is as informative as the SAT witness: it proves the non-existence of a solution. This property is central to the cooperation detection mechanism developed in Section 5, where UNSAT under a modified encoding is used to certify that no solution exists without the cooperative interaction.

In this thesis, propositional variables are introduced to represent agent positions, laser beam states, and laser activity at each time step. The constraints of the bounded-horizon solvability problem are then encoded as CNF clauses, and a SAT solver is used as a decision oracle. The solver used throughout is Minisat22, a descendant of the MiniSat solver of Eén and Sörensson [22], accessed through the PySAT interface [23].

3.3 Problem Formalization

We now define the semantic objects and decision problems used throughout the thesis. Two properties are central. **Solvability** asks whether there exists a valid joint trajectory that brings all agents to their exits within a bounded horizon. **Cooperation requirement** asks whether every such trajectory must rely on the same-colour beam-truncation mechanism. Both are formally decidable, and both are certified by the solver for every accepted level.

This section operates at the level of the game semantics; the SAT encoding of these objects follows in Section 4.

The LLE environment supports between 1 and 4 agents. Accordingly, throughout this thesis we assume that $1 \leq n_a \leq 4$. This bound is an artifact of the LLE engine, not a limitation of the SAT encoding developed in Section 4; the encoding itself remains correct for any finite n_a.

Definition 3.1 (LLE Level)

An LLE level is a tuple $L = (H, W, C, s, \mathcal{W}, \mathcal{S}, \mathcal{E})$ where:

- $H, W \in \mathbb{N}^+$ are the height and width of the grid.
- $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ is the set of all grid positions. We use the image-coordinates convention: x is the column index and y is the row index, so that moving north decreases y and moving east increases x. The implementation uses the transposed naming $(i, j) = (y, x)$ but the SAT semantics are identical.
- $C = \{0, 1, ..., n_a - 1\}$ is the set of agent colours, with $1 \leq n_a \leq 4$.
- $s : C \to P$ assigns an initial position to each agent, and s is injective.
- $D = \{N, S, E, W\}$ is the set of laser directions.
- $\mathcal{W} \subseteq P$ is the set of wall positions.
- $\mathcal{S} \subseteq C \times D \times P$ is the set of laser sources. A source $(c, d, p) \in \mathcal{S}$ emits a laser of colour c in direction d from position p.
- $\mathcal{E} \subseteq P$ is the set of exit positions, with $|\mathcal{E}| = |C|$.
- In the instances studied in this thesis, each colour appears in at most one laser source.
- The sets $s(C)$, $\mathcal{W}$, $\mathcal{E}$, and

$$P_{\text{src}} = \{p \in P \mid \exists c \in C, d \in D : (c, d, p) \in \mathcal{S}\}$$

are pairwise disjoint.

We also write

$$C_{\text{src}} = \{c \in C \mid \exists d \in D, p \in P : (c, d, p) \in \mathcal{S}\}$$

for the set of colours that actually have a source.

Definition 3.2 (Active Beams)

Fix a time step t and a joint position map $p_t : C \to P$.

For a source $(c, d, p_s) \in \mathcal{S}$, consider the grid ray starting at p_s and extending in direction d.

- Under the **standard beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall or the cell occupied by the unique agent of colour c (if any such cell lies on the ray).
- Under the **strict beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall.

A laser of colour c is active at a position when the beam of the unique source of colour c is active there.

Definition 3.3 (Valid Joint Trajectory)

A **horizon** $T_{\max} \in \mathbb{N}$ is the maximum number of joint moves allowed in a candidate solution.

A **joint trajectory** of length $T_{\max}$ is a sequence of joint positions $\sigma = \left(p_0, p_1, ..., p_{T_{\max}}\right)$ where each $p_t : C \to P$ assigns a position to every agent at time step t.

A joint trajectory is **valid** if it satisfies the following conditions:

- **Initialization:** $p_0(c) = s(c)$ for all $c \in C$.
- **Movement:** For all $t < T_{\max}$ and all $c \in C$, $p_{t+1}(c)$ is reachable from $p_{t(c)}$ in one step: the agent may stay in place or move to a 4-neighbouring cell, but it may not move into a wall or a laser-source cell.
- **No collision:** $p_{t(c_1)} \neq p_{t(c_2)}$ for all t and all distinct $c_1, c_2 \in C$.
- **Conservative hand-off rule:** For all $t < T_{\max}$ and all distinct $c_1, c_2 \in C$, agents may not enter a cell occupied by another agent at the previous time step; that is, $p_{t+1}(c_1) \neq p_{t(c_2)}$ and $p_{t(c_1)} \neq p_{t+1}(c_2)$.
- **Laser safety:** No agent c_1 occupies a cell at time t where a laser of colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is active under the standard beam semantics of Definition 3.2.
- **Stay on exit:** If $p_{t(c)} \in \mathcal{E}$ for some $t < T_{\max}$, then $p_{t+1}(c) = p_{t(c)}$.

Definition 3.4 (Solvability)

A level L is **solvable** with horizon $T_{\max}$ if there exists a valid joint trajectory σ of length $T_{\max}$ such that the set of occupied positions at time $T_{\max}$ is exactly the set of exits:

$$\left\{p_{T_{\max}}(c) \mid c \in C\right\} = \mathcal{E}$$

A level is **solvable** without qualification if it is solvable for some finite horizon.

The restriction to a bounded horizon is natural for the SAT encoding. In the model studied here, beam activity at time t is a deterministic function of the joint position map p_t. Hence the full state is determined by the joint agent positions. If a trajectory repeats the same joint position twice, the intervening segment forms a loop and can be removed without affecting reachability of later states.

Therefore, if a level is solvable at all, it is solvable within a finite horizon bounded by the number of collision-free joint configurations.

Scope note. All formal guarantees in this thesis are stated relative to a fixed finite horizon $T_{\max}$. The solver decides whether a level is solvable within that horizon, not whether it is solvable under an unbounded notion of play. In practice, $T_{\max}$ is set to a value known to be sufficient for the instances of interest; the choice of horizon is a parameter of the generation process, not a formal limitation of the approach.

Definition 3.5 (Bounded-Horizon LLE Solvability Problem)

The **bounded-horizon LLE solvability problem** is the following decision problem.

- **Input:** an LLE level L and a horizon $T_{\max} \in \mathbb{N}$.
- **Question:** does L admit a valid joint trajectory of length $T_{\max}$ whose final occupied positions are exactly the exits?

Definition 3.6 (Strict Beam Semantics)

A **strict trajectory** of length $T_{\max}$ is defined exactly as in Definition 3.3, except that beam activity is computed using the strict beam semantics of Definition 3.2: same-colour occupancy no longer truncates the corresponding beam.

In the model studied here, the relevant cooperative action is precisely this same-colour beam-truncation mechanism: an agent occupies a position that would otherwise allow its own beam to continue, thereby opening a path for another agent.

Definition 3.7 (Cooperation Requirement with Horizon $T_{\max}$)

A level L is said to **require cooperation** with horizon $T_{\max}$ if:

- L is solvable under the standard semantics.
- L admits no strict trajectory of length $T_{\max}$ whose final positions occupy all exit tiles.

When the horizon is clear from context, we simply say that L requires cooperation. This bounded form is the one used throughout the rest of the methods chapter, since both the SAT solver and the cooperation detector operate at a fixed finite horizon.

Chapter 4

SAT-based Solver

4.1 Definition of Sets

We reuse the level objects introduced in Section 3.3: the grid dimensions H and W, the position set P, the colour set C, the direction set D, the wall set $\mathcal{W}$, the source set $\mathcal{S}$, the exit set $\mathcal{E}$, and the start map s.

The SAT reduction introduces a finite horizon $T_{\max} \in \mathbb{N}$, which is the maximum number of joint moves allowed in the bounded decision problem, together with the discrete time-step set $T = \{0, 1, ..., T_{\max}\}$.

The sets P_{src} and C_{src} are as defined in Section 3.3. We recall that each colour appears in at most one laser source; under this assumption, when a source of colour c exists, its position and direction are uniquely determined by c. We write $s(c)$ for the initial position of agent $c \in C$, as defined in Definition 3.1.

4.2 Definition of Variables

We introduce three families of propositional variables.

- $a_{c,x,y,t}$: true iff agent $c \in C$ occupies position $(x, y) \in P$ at time step $t \in T$.
- $b_{c,d,x,y,t}$: true iff the beam emitted by the source $(c, d, p_s) \in \mathcal{S}$ is active at position $(x, y) \in P$ at time $t \in T$.
- $l_{c,x,y,t}$: true iff a laser of colour $c \in C_{\text{src}}$ is active at position $(x, y) \in P$ at time $t \in T$.

4.3 Constraints and Logical Encoding

We now formalise the constraint families used in the reduction. Each group of clauses corresponds to one logical component of the bounded-horizon decision problem.

1. **Initialization**

   - **Agents:**
     Each agent $c \in C$ is placed at its designated starting position $s(c)$ at $t = 0$; all other positions are unoccupied by that agent:

$$\bigwedge_{c\in C}\bigwedge_{(x,y)\in P}\begin{cases} a_{c,x,y,0} & \text{if } (x,y)=s(c)\\ \neg a_{c,x,y,0} & \text{otherwise}\end{cases}$$

- **Laser sources:**
  For each laser source $(c,d,(x_s,y_s))\in\mathcal{S}$, the beam is always active at its origin, at every time step:

$$\bigwedge_{(c,d,(x_s,y_s))\in\mathcal{S}}\bigwedge_{t\in T} b_{c,d,x_s,y_s,t}$$

2. **Agent Movements**

We define the set of positions reachable from (x,y) in one step as (x,y) itself together with its four grid neighbours, excluding walls and laser-source positions:

$$\begin{aligned}\text{next}(x,y)=\{(x',y')\in\{(x,y),(x,y-1),(x+1,y),(x,y+1),(x-1,y)\}\mid\\ (x',y')\in P,(x',y')\notin\mathcal{W},(x',y')\notin P_{\text{src}}\}\end{aligned}$$

- **Forward consistency:**
  If agent c is at position (x,y) at time t, it must be at some position in $\text{next}(x,y)$ at time $t+1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t}\rightarrow\bigvee_{(x',y')\in\,\text{next}(x,y)} a_{c,x',y',t+1}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\neg a_{c,x,y,t}\vee\bigvee_{(x',y')\in\,\text{next}(x,y)} a_{c,x',y',t+1}$$

Two methods are proposed to enforce that each agent occupies at most one position at each time step.

- **Global uniqueness:**
  No two distinct positions can both be occupied by agent c at time t:

$$\bigwedge_{c\in C}\bigwedge_{t\in T}\bigwedge_{(x_1,y_1)\in P}\bigwedge_{\substack{(x_2,y_2)\in P,\\(x_2,y_2)\neq(x_1,y_1)}}\neg a_{c,x_1,y_1,t}\vee\neg a_{c,x_2,y_2,t}$$

- **Local uniqueness:**
  No two distinct positions in $\text{next}(x,y)$ can simultaneously be occupied by agent c at time $t+1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\bigwedge_{(x',y')\in\,\text{next}(x,y)}\bigwedge_{\substack{(x'',y'')\in\,\text{next}(x,y),\\(x'',y'')\neq(x',y')}}\neg a_{c,x',y',t+1}\vee\neg a_{c,x'',y'',t+1}$$

Backward consistency:
If agent c is at position (x,y) at time $t+1$, it must have been at some position in $\text{next}(x,y)$ at time t:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t+1}\rightarrow\bigvee_{(x',y')\in\,\text{next}(x,y)} a_{c,x',y',t}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\neg a_{c,x,y,t+1}\vee\bigvee_{(x',y')\in\,\mathrm{next}(x,y)}a_{c,x',y',t}$$

- **No simultaneous occupation:**
  Two distinct agents cannot share the same position at the same time, nor can they move to a position already occupied by the other agent:

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t\in T}\neg a_{c_1,x,y,t}\vee\neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1}\neg a_{c_1,x,y,t+1}\vee\neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C,c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1}\neg a_{c_1,x,y,t}\vee\neg a_{c_2,x,y,t+1}$$

- **Victory condition:**
  Each exit must be occupied by at least one agent at time $T_{\max}$:

$$\bigwedge_{(x,y)\in\mathcal{E}}\bigvee_{c\in C}a_{c,x,y,T_{\max}}$$

  Combined with the no-collision clauses below and the equality $|\mathcal{E}|=n_a$, this clause family yields a bijection between agents and exits at time $T_{\max}$: the disjunctions force a surjection from C onto $\mathcal{E}$, and a surjection between two finite sets of equal size is automatically a bijection.

- **Stay on exit:**
  Once an agent reaches an exit, it remains there for all subsequent time steps:

$$\bigwedge_{c\in C}\bigwedge_{(x,y)\in\mathcal{E}}\bigwedge_{t=0}^{T_{\max}-1}\neg a_{c,x,y,t}\vee a_{c,x,y,t+1}$$

3. **Laser Activity**

   We define $\mathrm{next}_{d(x,y)}$ as the position immediately adjacent to (x,y) in direction d:

$$\mathrm{next}_{d(x,y)}=\begin{cases}(x,y-1)\text{ if }d=N\\(x+1,y)\text{ if }d=E\\(x,y+1)\text{ if }d=S\\(x-1,y)\text{ if }d=W\end{cases}$$

   - **Walls block beams:**
     A beam cannot be active at a wall position:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}}\bigwedge_{(x,y)\in\mathcal{W}}\bigwedge_{t\in T}\neg b_{c,d,x,y,t}$$

   - **Beam propagation:**
     Beam propagation clauses are instantiated only when the successor cell $(x',y')=\mathrm{next}_{d(x,y)}$ lies inside the grid, is not a wall, and is not itself a source cell. Source cells are handled separately by the initialization clauses above. Under these conditions, the beam is active at (x',y') iff it is active at (x,y) and no agent of colour c occupies (x',y'):

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\notin P_{\mathrm{src}}}} b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right)$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\notin P_{\mathrm{src}}}} \neg b_{c,d,x,y,t} \vee a_{c,x',y',t} \vee b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\notin P_{\mathrm{src}}}} b_{c,d,x,y,t} \vee \neg b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \mathrm{next}_{d(x,y)}\notin P_{\mathrm{src}}}} \neg a_{c,x',y',t} \vee \neg b_{c,d,x',y',t}$$

- **Link between beam and laser variables:**
  Since each colour has at most one source in the instances considered here, the laser variable $l_{c,x,y,t}$ is true at a position iff the beam of the unique source of colour c is active there:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} b_{c,d,x,y,t} \leftrightarrow l_{c,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \left(\neg b_{c,d,x,y,t} \vee l_{c,x,y,t}\right) \wedge \left(\neg l_{c,x,y,t} \vee b_{c,d,x,y,t}\right)$$

- **Agents cannot step on active lasers:**
  An agent of colour c_1 cannot occupy a position where an active laser of some source colour $c_2 \in C_{\mathrm{src}}$, with $c_2 \neq c_1$, is present. Agents are immune only to lasers of their own colour:

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C_{\mathrm{src}},\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} l_{c_2,x,y,t} \rightarrow \neg a_{c_1,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C_{\mathrm{src}},\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \neg l_{c_2,x,y,t} \vee \neg a_{c_1,x,y,t}$$

4.4 Clause Complexity and Polynomial Size

To show that the reduction has polynomial size, it is enough to bound the number of generated clauses as a function of the input parameters. Let

$$
\begin{aligned}
n &= |C|, \\
p &= |P| = HW, \\
s &= |\mathcal{S}|, \\
e &= |\mathcal{E}|, \\
\tau &= T_{\max} + 1
\end{aligned}
$$

and write

$$V = P \setminus (\mathcal{W} \cup P_{\mathrm{src}})$$

for the walkable cells.

We count **clauses** rather than **literals**. A CNF formula is a conjunction of clauses, each clause being a disjunction of literals. The total literal count is at most a constant factor larger than the clause count in our encoding, since every clause family generated below contains at most $\max(|\mathrm{next}(u)| + 1, n_a, 4) \leq 6$ literals per clause. Counting clauses is therefore sufficient to establish a polynomial bound on the formula size, and the same bound transfers to literals up to a constant factor. The main families admit the following clause bounds.

- **Initialization.** The agent-initialisation clauses contribute exactly np unit clauses, and the laser-source initialisation contributes exactly $s\tau$ unit clauses.
- **Movement constraints.** Forward consistency contributes at most $n(\tau-1)p$ clauses. The two uniqueness formulations differ here:

  $$\text{global uniqueness} = n(\tau-1)\binom{p}{2}$$

  $$\text{local uniqueness} = n(\tau-1)\sum_{u \in V}\binom{|\mathrm{next}(u)|}{2}$$

  Since $|\mathrm{next}(u)| \leq 5$, the local uniqueness term is bounded by

  $$n(\tau-1)10\ |V| \leq 10n(\tau-1)p$$

  and backward consistency contributes at most $n(\tau-1)p$ further clauses. The collision and conservative hand-off clauses contribute at most

  $$\binom{n}{2}(\tau + 2(\tau-1))p = \binom{n}{2}(3\tau-2)p$$

  The exit condition contributes exactly e clauses, and the stay-on-exit condition contributes exactly $ne(\tau-1)$ clauses.
- **Laser constraints.** The laser-safety clauses contribute at most $ns\tau p$ clauses. Beam propagation contributes at most $3s\tau p$ clauses, because each admissible propagation edge yields either one wall-blocking clause or three equivalence clauses. The beam/laser linking clauses contribute exactly $2s\tau p$ clauses.

Therefore the total number of clauses is polynomial in n, p, s, and τ. More precisely, with the global formulation the dominant term is

$$O(n\tau p^2 + n^2\tau p + ns\tau p)$$

while with the local formulation it is

$$O(n\tau p + n^2\tau p + ns\tau p)$$

Since each clause is generated by a simple bounded computation inside these loops, the reduction itself is computable in polynomial time as well. This justifies the claim that bounded-horizon LLE solvability is polynomial-time reducible to SAT.

4.5 Correctness of the Reduction

Proposition 4.8 (Correctness of the SAT Reduction)

Let L be an LLE level and let $T_{\max}$ be a horizon. For either movement formulation described above, the CNF formula $\Phi(L, T_{\max})$ is satisfiable if and only if there exists a valid joint trajectory of length $T_{\max}$ for L.

Proof.

For soundness, assume $\Phi(L, T_{\max})$ is satisfiable. Initialization fixes exactly one start position for each agent at time 0. For the global formulation, forward consistency together with pairwise exclusion ensures by induction on time that each agent occupies exactly one legal position at every later step. For the local formulation, the same conclusion follows from forward consistency, local exclusivity, and backward consistency. We may therefore define a joint trajectory $p_{t(c)}$ by reading off the unique true agent variable for each colour c and time t. The movement clauses enforce legal motion between consecutive steps; the collision clauses enforce both simultaneous separation and the conservative hand-off rule; the laser clauses enforce safety with respect to active beams; and the exit clauses enforce the terminal condition. Hence the extracted trajectory is valid.

For completeness, assume a valid joint trajectory of length $T_{\max}$ is given. Set each $a_{c,x,y,t}$ according to whether agent c occupies (x, y) at time t in the trajectory. Set beam variables $b_{c,d,x,y,t}$ and laser variables $l_{c,x,y,t}$ according to the deterministic beam dynamics induced by the same agent positions. Every clause family is then satisfied by construction: initialization matches the start state, movement and collision clauses match the trajectory semantics, beam propagation follows the induced laser dynamics, and the final positions occupy all exits. Therefore $\Phi(L, T_{\max})$ is satisfiable. □

4.6 Complexity-Theoretic Consequences

We can now state the consequence for the decision problem introduced in Definition 4.5.

The bounded-horizon LLE solvability problem lies in **NP**. A candidate trajectory can be verified in polynomial time by simulating the joint execution and checking that all agents occupy the exit tiles at the end without violating the movement, collision, and laser constraints defined in Section 3.3.

Combined with the polynomial-time construction above and Proposition 4.8, this shows that bounded-horizon LLE solvability is polynomial-time many-one reducible to SAT:

$$\text{LLE-Solvability} \leq_p \text{SAT}$$

Thus bounded-horizon LLE solvability is **at most as hard as SAT**: any SAT algorithm yields an algorithm for this decision problem with only polynomial overhead from the reduction.

Whether bounded-horizon LLE solvability is also **NP-hard** remains open in the present work. Establishing NP-hardness would require a polynomial-time reduction in the opposite direction, from a known NP-hard problem to LLE solvability. This thesis does not claim such a result.

It is also important to distinguish proved statements from standard complexity-theoretic beliefs. The question whether $\mathrm{P} = \mathrm{NP}$ remains open. Accordingly, statements here about worst-case difficulty should be read only through the formal claims we have established: SAT is NP-complete, bounded-horizon LLE solvability belongs to NP, and the reduction above places bounded-horizon LLE solvability within the polynomial-time many-one image of SAT.

In practice, this positioning explains why a SAT-based approach is attractive: the solver inherits the strong empirical performance of modern CDCL SAT solvers on many structured instances, even though the worst-case complexity remains exponential.

Chapter 5

Cooperation Detection

5.1 Strict SAT Encoding

Recall from Definition 3.6 that strict beam semantics changes only one aspect of the dynamics: same-colour occupancy no longer truncates the corresponding beam. Same-colour immunity is unchanged.

Accordingly, the strict SAT encoding keeps the standard laser-safety clauses and replaces only the same-colour beam-propagation rule. For every source $(c, d, p_s) \in \mathcal{S}$, every admissible propagation edge from (x, y) to (x', y'), and every time step $t \in T$, the strict encoding uses

$$b_{c,d,x',y',t} \leftrightarrow b_{c,d,x,y,t}$$

instead of the standard equivalence

$$b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right).$$

Thus the beam continues through agents of the matching colour instead of stopping at them. We denote the resulting CNF formula by $\Phi_{\text{strict}}(L, T_{\max})$ and the corresponding solver by StrictSolver.

5.2 Why This Captures Cooperation

Under the LLE mechanics studied here, the cooperative action of interest is for an agent to occupy a cell that would otherwise allow its own beam to continue, thereby making another agent's path safe. Standard solvability allows this beam-truncation mechanism; strict solvability removes it.

Therefore, if a level is solvable under the standard semantics but unsatisfiable under the strict semantics, every successful standard solution must rely on at least one same-colour beam-truncation step.

5.3 Formal Theorem and Proof

Theorem 4.9 (Cooperation Detection Criterion)

Let L be an LLE level and $T_{\max}$ a time horizon. Then L requires cooperation with horizon $T_{\max}$ if and only if $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

Proof.

($\rightarrow$) Assume that L requires cooperation with horizon $T_{\max}$. By Definition 3.7, L is solvable under the standard semantics, so $\Phi(L, T_{\max})$ is satisfiable. Suppose for contradiction that $\Phi_{\text{strict}}(L, T_{\max})$ is also satisfiable. Then there exists a strict trajectory whose final positions occupy all exit tiles. Since strict beam semantics differs from the standard one only by removing same-colour beam truncation, such a trajectory is also a valid standard trajectory that succeeds without using that mechanism. This contradicts Definition 3.7. Therefore $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

($\leftarrow$) Assume that $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable. The first condition implies that L is solvable under the standard semantics. Suppose that some successful standard trajectory used no same-colour beam-truncation step. Then the same joint positions would also satisfy the strict beam semantics, because the only semantic difference between the two models concerns exactly that truncation mechanism. This would yield a satisfying assignment for $\Phi_{\text{strict}}(L, T_{\max})$, contradicting unsatisfiability. Hence every successful standard trajectory must use at least one same-colour beam-truncation step, so L requires cooperation with horizon $T_{\max}$. □

5.4 Practical Algorithm

The cooperation detector runs two SAT calls on the same level:

1. Run $\text{Solver}(L, T_{\max})$. If the result is UNSAT, the level is unsolvable for that horizon, so it is rejected before cooperation is considered.
2. Run $\text{StrictSolver}(L, T_{\max})$. If the result is UNSAT, the level requires cooperation for the same horizon.

Both calls share the same bounded horizon and differ only in the beam-propagation clauses. For benchmark levels, the horizon can be chosen from known solution lengths; for generated levels, it is the user-supplied generation parameter $T_{\max}$.

Scope note. The cooperation notion defined here is intentionally specific: it captures same-colour beam-truncation as the relevant cooperative act. A level that requires two agents to coordinate their movements for unrelated geometric reasons — without any laser blocking being involved — would not be identified as cooperative by this detector. This definition is not claimed to exhaust every possible interpretation of cooperation in multi-agent environments; it is the specific mechanism studied in this benchmark, and the formal guarantee is scoped accordingly.

Model-dependence of finer-grained analyses. The binary detector above is a property of the **level**: the satisfiability of $\Phi(L, T_{\max})$ and $\Phi_{\text{strict}}(L, T_{\max})$ does not depend on which satisfying assignment the SAT solver happens to return. Any finer-grained classification of cooperation, however — such as the cooperation profile analyzer introduced in Section 6, which builds a directed dependency graph between agents from a single extracted joint plan — does depend on the specific model returned. A cooperative level typically admits many valid joint plans, and different plans may exhibit different helping patterns: an agent that helps another in one solution may be passive in another. Consequently,

the profile label attached to a level reflects the structure of the extracted plan, not an intrinsic invariant of the level. This is acceptable for the generation use case considered here — the analyzer still acts as a sound filter that certifies the extracted plan exhibits the targeted profile — but two solver runs on the same level can in principle yield different profile labels.

Chapter 6

Procedural Generators

6.1 Design Pattern

All generators follow a common architecture built around three principles:

1. **SAT as oracle**: the solver is not post-hoc; it is embedded in the generation loop. A candidate level is accepted only if the solver confirms the desired property (solvability, cooperation).
2. **Separation of concerns**: level construction and property verification are decoupled. Generators build candidate levels using domain-specific heuristics; the solver decides acceptance.
3. **Extensibility**: every generator extends `BaseGenerator` and is registered via the `@register_generator` decorator, making it available to the CLI without modifying core code.

In implementation terms, each generator repeatedly performs the following loop: sample or construct a candidate layout, reject it if it violates generator-specific structural constraints, build an `lle.World`, and finally run the appropriate SAT-based acceptance test. Solvable generators stop after the first candidate certified satisfiable within the target horizon, while cooperative generators add the strict-beam counterfactual test and, optionally, a cooperation-profile filter.

6.2 Generation Targets

Viewed through the solvability and cooperation definitions of Section 3.3, the generator family targets the three level categories shown in Figure Figure 2. Solvable generators accept levels in categories (b) and (c), cooperative generators accept only levels in category (c), and unsolvable levels in category (a) are always rejected.

(a) Unsolvable
rejected by all generators

(b) Solvable, no cooperation
accepted only by solvable generators

(c) Solvable and cooperative
target of cooperative generators

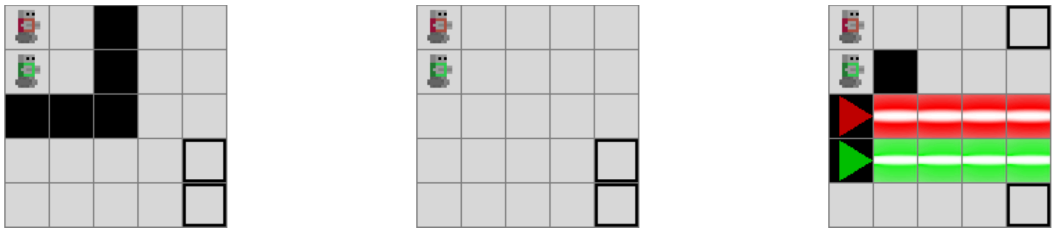

Figure 2: Target level categories for the generator family.

6.3 Random Solvable Generator

The random solvable generator is the baseline member of the family. It samples pairwise-distinct positions for agent starts, exits, walls, and laser sources uniformly over the grid, assigns a random direction to each source, and submits the resulting world to the solver. A candidate is accepted only if it is satisfiable within the requested horizon $T_{\max}$; when a lower bound $T_{\min}$ is provided, the generator also requires the candidate to be unsatisfiable for $T_{\min} - 1$, thereby selecting levels that fall inside a difficulty window.

This generator is deliberately simple. Its main value is methodological: it gives an unbiased sampling baseline against which more structured generators can be compared. Its main weakness is rejection rate. As the grid grows and the number of interacting entities increases, purely random layouts quickly become dominated by unsolvable or trivial instances.

6.4 Constrained Random Solvable Generator

A structured variant that biases generation toward solvable configurations before any SAT call is made. Relative to the random solvable generator, it rejects candidates that are already geometrically degenerate, for example when a laser points outside the grid immediately, when a laser would have zero beam length, or when an exit lies on an unavoidable beam segment.

These filters do not themselves prove solvability, but they remove a large class of obviously bad candidates before invoking the solver. The generator therefore remains sound with respect to the formal solvability guarantee, while typically spending less time on layouts that fail for purely local geometric reasons.

6.5 Random Cooperative Generator

The random cooperative generator extends the random solvable generator with a second SAT test based on the strict semantics of Section 5. A candidate is accepted only if it is satisfiable under the standard encoding and unsatisfiable under the strict encoding. This guarantees that every accepted level structurally requires the beam-truncation mechanism identified as cooperation in Definition 3.7.

The current implementation augments this binary guarantee with a **cooperation profile analyzer**. The binary detector remains the formal guarantee used throughout the thesis: a level is cooperative if and only if it is satisfiable under the standard semantics and unsatisfiable under the strict semantics. The analyzer adds a second layer whose purpose is to distinguish **which kind** of cooperation the accepted level exhibits.

Given a cooperative level, we first extract a valid joint plan from the standard SAT model. We then run selective counterfactual checks in which one agent at a time loses the same-colour laser interaction used for helping behaviour. If the level becomes unsatisfiable under that selective restriction, the agent is identified as a necessary helper. By combining these counterfactual checks with the helping actions observed in the extracted plan, we build a directed dependency graph between agents.

This dependency graph is used as a generation target. In the present implementation, the generator can recognise and filter levels according to the following profile families:

- **cooperative**: binary cooperation is required, regardless of finer structure;
- **asymmetric**: at least one one-way helping relation is present;
- **mutual**: two agents depend on each other;
- **chain**: dependencies form a directed chain without branching;
- **distributed**: one agent depends on multiple distinct helpers;
- **fully coupled**: all agents belong to a single strongly connected dependency component.

The important methodological point is that profile control is layered on top of the existing formal machinery. The SAT encodings still certify solvability and binary cooperation. The profile analyzer uses those certified levels as input and acts as a classification and filtering layer for the generator.

6.6 Constrained Random Cooperative Generator

The constrained random cooperative generator combines the geometric filters of the constrained solvable generator with the binary cooperation test and optional profile filter of the random cooperative generator. In other words, it first avoids immediately degenerate geometries, then requires the surviving candidates to satisfy the same solver-based cooperation criterion.

This generator therefore targets the same formally certified output class as the random cooperative generator, but with a sampling distribution biased away from trivial failures. It is useful when the goal is not only to obtain cooperative levels, but to obtain them with fewer discarded samples.

6.7 Constructive Solvable Generator

The constructive solvable generator replaces blind sampling with a partial-by-construction layout. On a grid of H rows and W columns with n_a agents, it picks a random orientation, samples a set of n_a distinct **lane indices** without replacement on the orientation axis (rows for the horizontal orientation, columns for the vertical one), places one agent start at one end of each lane and the corresponding exit at the other end, and reserves every cell of every lane as non-buildable. Walls and lasers are sampled only from the remaining cells, with walls drawn from a uniformly shuffled list of free cells and truncated to the requested wall budget. Additional lasers are accepted only if their full beam segment avoids every reserved cell. The solver acts as the final verifier, but the sampling process is strongly biased toward jointly solvable instances. Lanes are sampled **without** the contiguity constraint used in

earlier prototypes, so the lane band can be split anywhere on the orientation axis, which is the main source of within-pool diversity.

6.8 Constructive Cooperative Generator

The constructive cooperative generator inherits the lane machinery of the constructive solvable generator and replaces its laser-placement step with one that plants a deliberate cooperation dependency for every laser, not just one. The geometry is built in the following sequence (see `src/generators/cooperative.py`).

1. **Orientation.** One of the two orientations (horizontal lanes / vertical lanes) is chosen uniformly at random per call, subject to feasibility constraints on the grid dimensions.
2. **Lane sample.** A set $L \subseteq \{0, \ldots, D-1\}$ of $|L| = n_a$ lane indices is sampled without replacement on the orientation axis ($D = H$ or $D = W$). The lanes are **not** required to be contiguous; any subset of size n_a is admissible.
3. **Rotation flip.** Independently of the orientation, a fair coin chooses whether the agent starts sit on the first or the last index of the perpendicular axis (and the exits sit on the opposite edge). Combined with the two orientations this gives all four rotations (agents on left, right, top, or bottom) with equal probability.
4. **Structural laser placement.** For each of the n_l requested lasers, the generator picks
   - a **perpendicular column** (or row, in the vertical orientation) from a pool of distinct values in the interior of the perpendicular axis: $\{1, \ldots, D_{\text{perp}} - 2\}$;
   - an axis position $a \notin L$ strictly before the lane band ($a < \min(L)$) or strictly after it ($a > \max(L)$);
   - a direction perpendicular to the lane axis, pointing toward the lane band.

   These choices are independent across the n_l lasers. Distinctness of the perpendicular columns guarantees that the resulting beams are parallel straight lines on disjoint columns, so no two laser sources or beam segments coincide. Each laser is given a **distinct colour** in $\{0, \ldots, n_l - 1\}$.
5. **Beam-path reservation.** The full unblocked beam segment of every laser, from source to grid edge, is added to the reserved cell set. Without this step, walls placed in the next step could land on a beam cell strictly between two non-adjacent lanes and clip the beam before it reaches the far lane, which would silently break the cooperation requirement.
6. **Walls.** The remaining free cells are shuffled uniformly and the first `num_walls` of them are placed as walls. Shuffling — rather than taking the row-major prefix — is what gives within-pool wall-mask diversity.
7. **SAT verification and profile filter.** The candidate is built into an `lle.World`, the standard and strict SAT encodings of Section 5 are run, and the cooperation profile analyzer is applied. The candidate is accepted only if it is satisfiable under the standard semantics and unsatisfiable under the strict one — the binary cooperation criterion of Theorem 4.9 — and if the profile analyzer's classification matches the requested family (default: `cooperative`, which accepts any same-colour beam-truncation requirement).

The distinct-colour multi-laser construction is the key qualitative change relative to a single-structural-laser template: with $n_l \geq 2$ each laser's beam can only be safely truncated by the unique agent of its colour, so cooperation involves n_l helpers acting on their own beams in turn, rather than a single helper acting on one beam. On the parameter configurations of the experiments in Section 7 this consistently

produces **mutual** profiles (every helper is also a beneficiary) for $n_l = 2$ and a mix of **mutual** and **distributed** profiles for $n_l \geq 3$ when the grid is large enough.

6.9 Constructive Level-6-Style Generator

The constructive level-6-style generator targets the specific layout shape of LLE Level 6: clustered agent starts and clustered exits placed on opposing sides of the grid, with a corridor of lasers between them. Agents are placed in a small rectangular cluster (a 2×2 block for four agents) in one third of the grid; exits are placed in a matching cluster in the opposite third. The orientation (vertical with start above exit, or horizontal with start left of exit) is chosen at random per call, and the perpendicular position of each cluster is also randomised, so that the two clusters are always forced to span at least a third of the grid in their main axis. Lasers are then placed inside the corridor between the clusters, oriented perpendicular to the cluster axis so that each beam crosses the corridor; remaining free cells receive walls up to the requested wall budget. This geometry consistently produces **mutual** cooperation profiles that mirror the structure of Level 6, and it is the generator we recommend for producing training instances intended to transfer to Level 6.

6.10 Summary

Generator	Construction Bias	Solvable	Cooperative
Random Solvable	Uniform random sampling	Yes (SAT check)	No
Constrained Random Solvable	Random + geometric rejection	Yes (SAT check)	No
Random Cooperative	Uniform random sampling	Yes (SAT check)	Yes (strict UNSAT)
Constrained Random Cooperative	Random + geometric rejection	Yes (SAT check)	Yes (strict UNSAT)
Constructive Solvable	Reserved agent lanes	Yes (SAT check)	No
Constructive Cooperative	Reserved lanes + planted dependency	Yes (SAT check)	Yes (strict UNSAT)
Constructive Level-6-Style	Clustered starts/exits + corridor lasers	Yes (SAT check)	Yes (strict UNSAT)

Table 1: Overview of the implemented generators and their guaranteed properties.

Chapter 7

Empirical Evaluation

7.1 Evaluation Protocol

The first benchmark implemented in this project isolates the SAT solver rather than the generators. Its goal is to compare the two movement formulations used in the encoding: the **local** formulation, which combines neighbourhood-based exclusivity with backward consistency, and the **global** formulation, which enforces uniqueness by pairwise exclusion over the whole grid.

7.1.1 Metrics

For each run, the benchmarking code records:

- the total number of clauses in the generated CNF;
- the clause count contributed by each major constraint family;
- the CNF generation time;
- the SAT solving time;
- the total time obtained by summing generation and solving.

The implementation also stores per-constraint method profiles for the movement constraint, which allows us to inspect the part of the final CNF generated inside the movement-constraint module.

7.1.2 Solver and Level Sets

All benchmark runs use the same SAT backend as the main solver implementation, namely `Minisat22` through the PySAT interface. The benchmarking script can evaluate the six hand-crafted benchmark levels distributed with the environment, each paired with a horizon known to be sufficient for solvability. It can also benchmark custom levels constructed programmatically.

The experiment reported in Section 7 uses four representative levels: three synthetic instances of increasing size and one original benchmark level. This combination exposes both scaling behaviour and the behaviour of the solver on a realistic cooperative puzzle.

7.1.3 Protocol

For each level and each movement formulation, the benchmark performs one profiled run to extract the exact clause counts and the full constraint breakdown. It then repeats the same solver invocation for 100 runs, each time on a fresh copy of the world, and reports the mean and standard deviation of generation time and solve time. Using fresh world copies avoids benchmark contamination by mutable environment state.

No timeout or parallel speedup is introduced in this protocol. The measurements should therefore be read as direct comparisons between the two SAT formulations on the same machine, rather than as hardware-independent absolute performance claims.

7.1.4 Outputs

The benchmarking pipeline produces a console summary table, a JSON file containing aggregated benchmark statistics, and a set of plots for clause counts, per-constraint clause breakdowns, and timing statistics. The results section (Section 7) draws on these outputs to interpret the trade-off between the two movement formulations.

The current experimental evaluation is intentionally focused. Its purpose is not yet to validate the full generator family, but to measure the effect of one modeling choice inside the SAT solver: the formulation of the agent-uniqueness constraint.

7.2 Experimental Question

The experiment asks whether the **global** and **local** movement formulations introduced in Section 4 differ materially in CNF size and runtime on levels of increasing complexity.

This is an important baseline question because the uniqueness constraint appears at every time step for every agent. A poor formulation therefore affects the full reduction, not only a negligible subpart of it. The protocol itself is defined in Section 7.1; the present chapter reports only the level set, the observed results, and their interpretation.

7.3 Benchmark Instances

Four levels of increasing size and complexity were used:

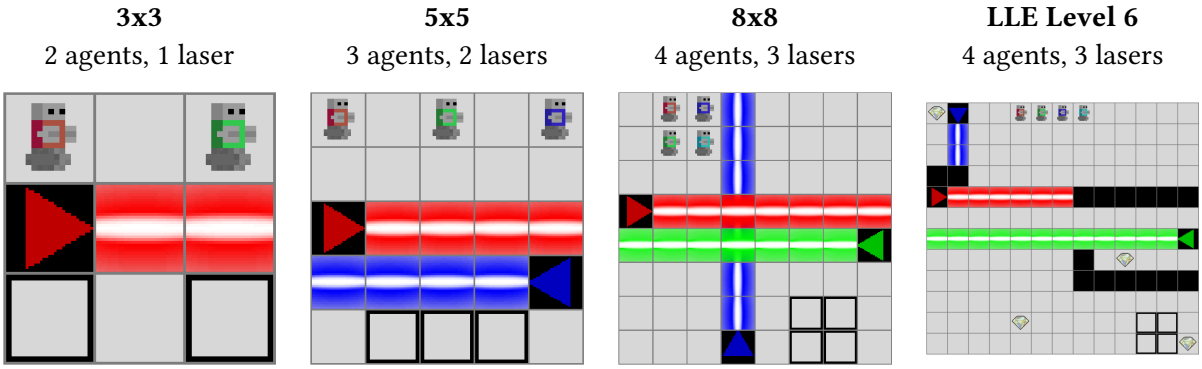

Figure 3: The four test levels used for benchmarking, ordered by increasing complexity.

The first three levels were constructed to expose scaling behaviour as grid size and agent count increase. The fourth is LLE Level 6, one of the hand-crafted benchmark levels distributed with LLE, included to assess solver behaviour on a realistic cooperative instance.

7.4 Results

7.4.1 Clause Count

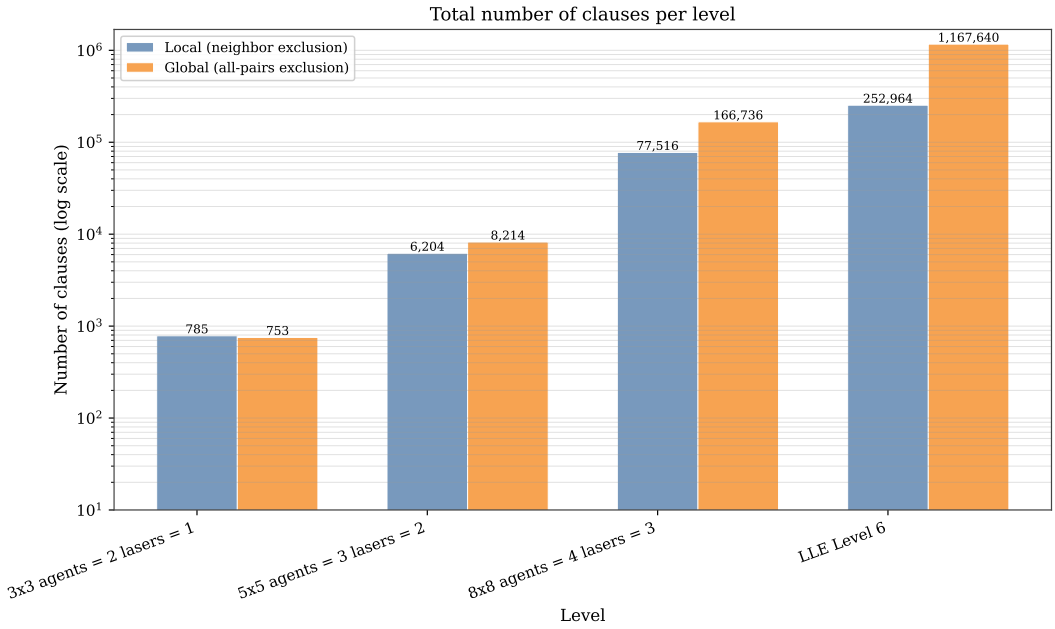

Figure 4: Number of clauses generated by the global and local uniqueness formulations across the four benchmark levels.

The clause-count results closely follow the theoretical expectation developed in Section 4. On the smallest 3×3 level, the global formulation is slightly more compact than the local one (753 clauses versus 785), which is consistent with the fact that the instance lies below the expected crossover regime. From the 5×5 level onward, the trend reverses: the local formulation generates 6,204 clauses versus 8,214 for the global one, then 77,516 versus 166,736 on the 8×8 level, and finally 252,964 versus 1,167,640 on LLE Level 6.

This widening gap is the main structural result of the experiment. The local formulation scales much better because its exclusivity clauses are tied to neighbourhood-sized successor sets, while the global formulation introduces pairwise exclusions across the full position set.

7.4.2 Solving Time

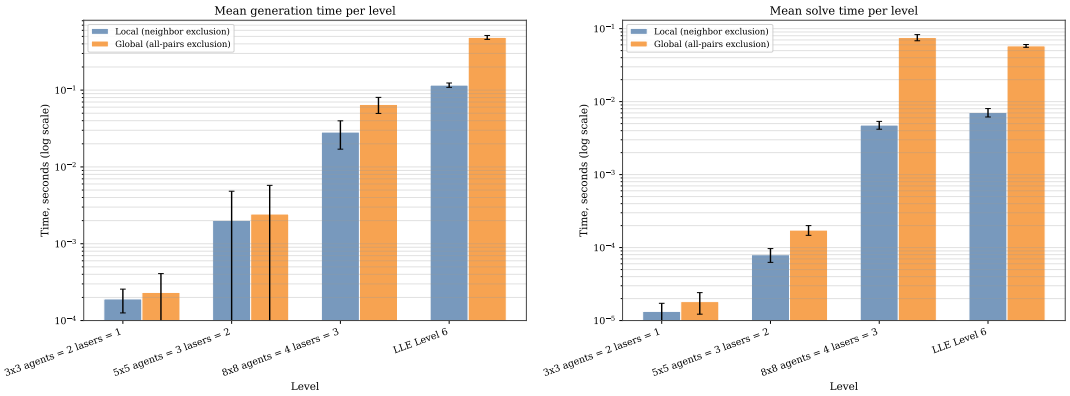

Figure 5: SAT solver time (seconds) for the global and local uniqueness formulations across the four benchmark levels.

The timing results show the same qualitative pattern. On the smallest synthetic instances, both methods solve essentially instantaneously and the difference is negligible. Once the levels become moderately large, however, the local formulation is consistently faster both to generate and to solve. The contrast is especially visible on the 8×8 level and on LLE Level 6, where the much larger global CNF induces a clear runtime penalty. Both axes are plotted in logarithmic scale because clause counts and runtimes span more than three orders of magnitude across the four levels.

One subtlety deserves comment. On LLE Level 6 the global formulation produces about 7 times more clauses than on the synthetic 8×8 instance, yet its mean solve time is **lower**. This is not a contradiction: SAT solver runtime is driven by the search structure, not strictly by formula size. Hand-crafted levels such as Level 6 admit propagation-friendly satisfying assignments that modern CDCL solvers find quickly, while randomly structured large instances can expose pathological search behaviour even at smaller clause counts.

The practical conclusion is therefore the same as the analytical one: the local formulation is not merely a cleaner theoretical encoding, but the preferable default implementation choice for the solver developed in this thesis.

7.5 Interpretation and Scope

The current results support one claim strongly and several others only weakly.

What is established is that the internal SAT formulation matters. The choice between local and global uniqueness has a first-order effect on both CNF size and runtime, and that effect is already substantial on realistic LLE instances.

What is **not** yet established empirically is the broader quality of the generator framework. The present experiment does not measure generator acceptance rates, diversity of accepted levels, distribution of cooperation profiles, or any downstream training effect on MARL agents. The following sections address the first two of these open questions.

7.6 Generator Rejection Rates

7.6.1 Experimental Question

Acceptance rates determine the practical cost of the rejection-sampling strategy used by the generators. A generator with a 1 % acceptance rate requires approximately one hundred solver calls per usable level, which directly affects generation throughput. This experiment measures, for each generator and grid size, how many attempts are needed to find one accepted level and how much time those attempts consume.

7.6.2 Protocol

For each generator type and grid size combination, a fixed number of independent trials is executed: 200 trials for 3×3 and 5×5 grids, and 20 trials for the 8×8 grid (see Section D.5 for full parameters). Each trial searches for one accepted level by calling the generator repeatedly with a budget of one attempt per call. A call either returns an accepted level or raises a rejection. The trial ends when one level is accepted or a per-trial budget is exhausted (500 attempts for small grids; 100 attempts or 30 seconds for 8×8). Failed trials — those that exhaust the budget — are recorded separately and excluded from the mean; they represent the hard tail of the attempt distribution.

Each single attempt samples a candidate layout, validates geometric constraints (where applicable), constructs an `lle.World` object, and runs the SAT solver. The attempt is accepted if the solver certifies the required property (solvability or cooperation), and rejected otherwise.

Recording the number of attempts per accepted level is the most direct measurement of the generator's rejection cost: the mean number of attempts approximates the reciprocal of the acceptance rate.

Five generators are evaluated: `constrained_random_solvable`, `constrained_random_cooperative`, `constructive_solvable`, `constructive_cooperative`, and `constructive_level6_style`. The plain random generators are excluded as they serve primarily as structural baselines; the constrained and constructive variants are the generators of practical interest. Three grid sizes are used: 3×3 with 2 agents and 1 laser, 5×5 with 3 agents and 2 lasers, and 8×8 with 4 agents and 3 lasers.

7.6.3 Results

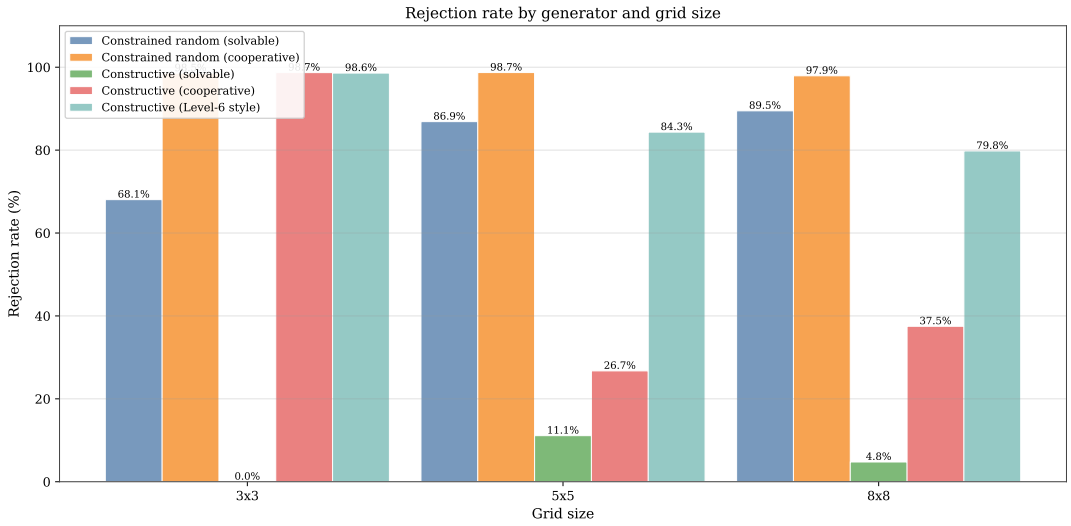

Figure 6: Rejection rate (%) per generator and grid size, derived from the mean number of attempts per accepted level across 50 trials.

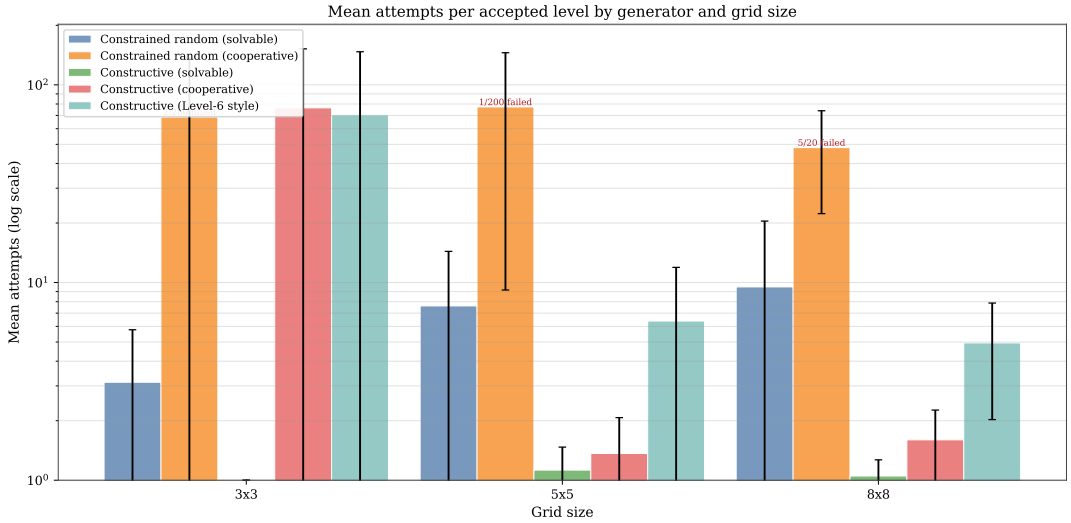

Figure 7: Mean number of attempts needed to find one accepted level, per generator and grid size. Each attempt is one independent generation call; the mean is averaged over 50 successful trials.

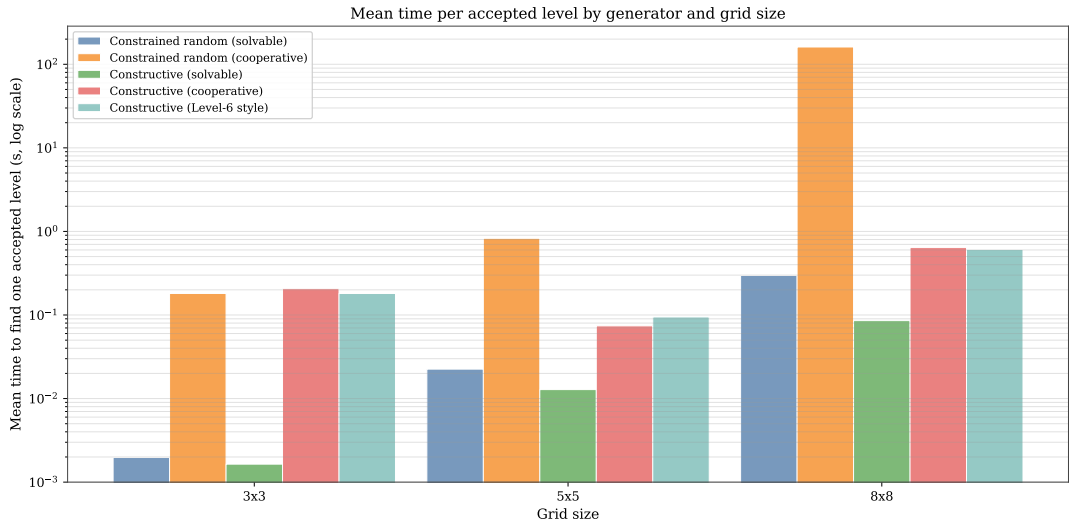

Figure 8: Mean wall-clock time (seconds) to obtain one accepted level, per generator and grid size. Time includes all rejected attempts that precede each accepted level.

7.6.4 Interpretation

The results reveal a four-way split rather than the expected two-way split between solvable and cooperative generators.

The `constrained_random_solvable` generator has rejection rates of 68 %, 87 %, and 89 % on the 3×3, 5×5, and 8×8 grids respectively, requiring a mean of 3.1, 7.6, and 9.5 attempts per accepted level. Rejection here is driven by the fact that random layouts rarely admit a valid joint trajectory within the requested horizon.

The `constructive_solvable` generator achieves near-zero rejection across all sizes: 0 % on 3×3, 11 % on 5×5, and 5 % on 8×8, with mean attempts of 1.0, 1.1, and 1.1. The lane-reservation strategy reliably biases layouts toward solvable configurations, making the acceptance check almost a formality.

The `constrained_random_cooperative` generator behaves consistently with the prior expectation: rejection rates of 98.5 %, 98.7 %, and 97.9 % across the three grid sizes, requiring 69, 77, and 48 mean attempts per accepted level. The high cost reflects the strict subset structure: cooperation-requiring layouts are a strict subset of solvable layouts, and a uniformly sampled candidate is unlikely to require beam-blocking.

The `constructive_cooperative` generator separates from this pattern. On 3×3 it shows the same expected high rejection (98.7 %, 76 mean attempts), because the grid is so small that even a deliberate dependency template rarely fits without geometric clashes. From 5×5 upward the planted-dependency strategy pays off: rejection drops to 27 % on 5×5 and 38 % on 8×8, with mean attempts of 1.4 and 1.6 respectively. This is a substantial qualitative result — at sizes large enough for the lane geometry to be feasible, the constructive cooperative template reliably plants a same-colour blocking dependency that the SAT-based cooperation test accepts on the first or second attempt.

The `constructive_level6_style` generator sits between the two cooperative families on this benchmark. It shows the same near-saturated rejection on 3×3 (98.6 %, 70 mean attempts) for the same geometric reason, but rejection rates of 84.3 % on 5×5 and 79.8 % on 8×8 (6.4 and 5.0 mean attempts respectively) — substantially higher than the lane-based `constructive_cooperative`, but materially lower than the random cooperative generator. The clustered geometry forces all agents through a

shared corridor, which makes solvability and cooperation requirement harder to satisfy simultaneously than in the lane-based template; in exchange, when a level is accepted, it carries a richer cooperation profile (see Section 7.7).

On the 8×8 grid, `constrained_random_cooperative` had 5 failed trials out of 20, where the per-trial budget (100 attempts or 30 seconds) was exhausted without finding an accepted level; `constructive_cooperative` had 0 failed trials. Failed trials are excluded from the mean-attempts statistic, which therefore under-estimates the true cost of `constrained_random_cooperative` on 8×8.

7.7 Cooperation Profile Distribution

7.7.1 Experimental Question

Binary cooperation detection certifies that every accepted cooperative level requires at least one same-colour beam-truncation step, but it does not distinguish which structural pattern of inter-agent dependency the level exhibits. This experiment measures the distribution of cooperation profiles among accepted levels, using the cooperation profile analyzer introduced in Section 5.

7.7.2 Protocol

For each of three cooperative generators (`constrained_random_cooperative`, `constructive_cooperative`, and `constructive_level6_style`) and two grid sizes (5×5 with 2 agents and 1 laser, 8×8 with 3 agents and 2 lasers), 100 cooperative levels are accepted on the small grid and 50 on the large grid, then classified by the cooperation profile analyzer. The classifier assigns each level to one of the following profile families: `asymmetric`, `mutual`, `chain`, `distributed`, or `fully_coupled`. The analyzer also returns `cooperative` as a fallback when cooperation is required but no specific dependency structure is detected.

7.7.3 Results

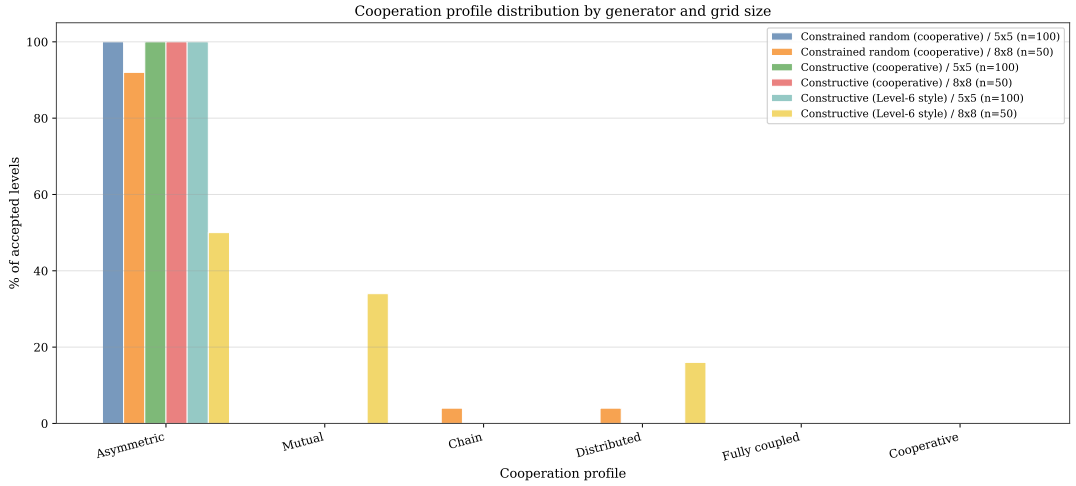

Figure 9: Distribution of cooperation profiles among accepted cooperative levels, grouped by generator and grid size. Each bar shows the fraction of accepted levels assigned to a given profile family.

7.7.4 Interpretation

Most generators produce a strongly asymmetric distribution. On the 5×5 grid, every accepted level from all three generators is classified as `asymmetric`: one agent depends on another, but not vice versa. The 5×5 configuration uses only 2 agents, so the profiles `chain`, `distributed`, and `fully_coupled` are **structurally impossible** by construction (they require at least 3 agents for chain/distributed and a non-trivial cycle of length ≥ 3 for `fully_coupled`). The 5 × 5 result is therefore a partition between `asymmetric` and `mutual`, and the absence of `mutual` levels indicates that random or constructive geometries on this grid rarely yield two reciprocally helping agents within the chosen horizon. On the 8×8 grid (3 agents, 2 lasers), the random cooperative generator produces 92 % `asymmetric`, 4 % `distributed`, and 4 % `chain` levels (n = 50), with no `mutual` or `fully_coupled` instances observed. The lane-based `constructive_cooperative` generator produces 100 % `asymmetric` levels on both grid sizes, which reflects its planted-dependency template: it deliberately introduces exactly one one-way helping relation.

The `constructive_level6_style` generator is the qualitative outlier. On 5×5 with 2 agents it is also forced to 100 % `asymmetric` for the structural reason above, but on 8×8 with 3 agents it produces a much more varied distribution: 50 % `asymmetric`, 34 % `mutual`, and 16 % `distributed` (n = 50). The `mutual` and `distributed` fractions are by far the highest of any generator in this benchmark, and they reflect the geometry of the level-6-style template: when all agents are forced through a shared corridor, cooperation tends to involve multiple agents helping each other simultaneously, rather than a single one-way dependency. This is the closest match in the generator family to the dependency structure of the canonical LLE Level 6 (which itself is mutual under the analyzer's classification cascade).

The concentration on `asymmetric` profiles in the other generators is consistent with the structural bias introduced by same-colour beam-blocking in lane-based or random layouts: the simplest cooperation pattern to arise is a one-way dependency, and richer profiles require either a more specific layout configuration or geometric pressure such as the shared-corridor constraint introduced by the level-6-style generator.

The lane-based constructive cooperative generator achieves 0 % rejection on the 5×5 grid (every generated candidate is accepted) and 19 % on the 8×8 grid, confirming that the lane-reservation strategy combined with the planted dependency reliably produces cooperative levels for the parameter combinations used here. The level-6-style constructive generator has higher rejection on this benchmark — 39 % on 5×5 and 44 % on 8×8 — but trades that moderate cost for the much richer profile distribution discussed above. By contrast, the random cooperative generator has 98 % rejection on 5×5 and 97 % on 8×8, because most randomly sampled layouts do not spontaneously require a beam-blocking step.

The profile distribution reveals a limitation of the current generation approach: although the larger random sample on 8×8 now exposes rare `chain` and `distributed` profiles (2 levels each out of 50), `mutual` and `fully_coupled` still do not appear in these samples. Generating levels with a specific rare profile reliably therefore still requires the profile-targeted generation mode, in which the generator accepts only levels that match the requested profile family.

7.8 Discussion

The three experiments above characterise complementary aspects of the generation framework.

The rejection-rate experiment measures **efficiency**: how many solver calls are needed per accepted level, and which generator and grid-size combinations are most practical. This is relevant for large-scale generation tasks where throughput matters.

The profile-distribution experiment measures **output diversity**: whether the accepted levels cover different cooperation structures or concentrate on a narrow profile class. This is relevant for curriculum design, where a varied distribution of cooperation structures is preferable to a homogeneous one.

Together, the three experiments support the claim that the solver-in-the-loop architecture produces not only formally certified levels, but a characterisable and controllable distribution of certified levels.

What these experiments do not establish is whether certified levels are useful for training. The generator framework guarantees formal properties of accepted levels, but whether those properties translate into better or faster learning for MARL agents compared to uncertified baselines remains an open empirical question, addressed in Section 7.9 and Section 7.10.

7.9 Learnability of Generated Cooperative Levels

7.9.1 Experimental Question

We now ask whether off-the-shelf MARL agents can learn to solve levels produced by the `cooperative` generator, and whether the choice of MARL algorithm matters as the cooperation requirement scales up in grid size and agent count. The cooperative generator certifies every accepted level under the binary criterion of Section 5: standard SAT and strict UNSAT, so every training and evaluation level requires at least one same-colour beam-truncation step. The learnability question is therefore not whether the levels are solvable in principle (they are, by construction), but whether the joint policy that solves them is reachable by common value-decomposition methods within a modest training budget.

We restrict ourselves to three baseline cooperative-MARL algorithms: independent Q-learning (IQL), value-decomposition networks (VDN), and QMIX. They form a natural progression in the strength of the credit-assignment assumption: from none (IQL), to additive team-value (VDN), to a monotonic mixing network (QMIX).

The experiment proceeds in two phases of increasing difficulty. Phase 1 fixes a small two-agent configuration that all three algorithms are expected to solve; its purpose is to establish a floor and to verify that the training pipeline and the generated levels are well-formed. Phase 2 scales up to three agents and two lasers, where credit assignment is expected to become the dominant difficulty.

7.9.2 Protocol

Each phase fixes a grid geometry, an agent and laser count, a horizon $T_{\max}$, and a generator. The two phases use the configurations summarised in Table 2, both drawn from the cooperative generator with seeded pre-flight pool generation (`PHASE1_GRID`, `PHASE2_GRID` in `src/experiments/learnability/configs.py`).

Phase	Grid	Agents	Lasers	$T_{\max}$	Generator	Steps
Phase 1	6×6	2	1	10	`cooperative`	100,000
Phase 2	8×8	3	2	16	`cooperative`	200,000

Table 2: Learnability-experiment configurations. The generator name refers to the registry key in `src/generators/registry.py`, which corresponds to the descriptive name `constructive_cooperative` used in Section 6.

For each phase, the pre-flight script generates two disjoint level pools from independent seeded streams: a training pool of $|\mathcal{D}_{\text{train}}| = 20$ levels (split seed 20260515) and a held-out test pool of $|\mathcal{D}_{\text{test}}| = 20$ levels (split seed 20260516). The same pools are reused for all algorithms and all training seeds within a phase, so any cross-algorithm difference is attributable to optimisation rather than to a pool resample. Per-level renderings of both pools are reproduced in the appendix (Section D.8, Section D.9).

For each phase we train $|\mathcal{A}| \times |\mathcal{S}| = 3 \times 20 = 60$ independent agent instances:

$$\mathcal{A} = \{\text{IQL}, \text{VDN}, \text{QMIX}\}, \quad \mathcal{S} = \{0, 1, \ldots, 19\}$$

The seed $s \in \mathcal{S}$ controls the Python, NumPy, and PyTorch random streams as well as the pool-sampling RNG. Hyperparameters are identical across algorithms and identical across phases (Section D.7): an Adam optimiser at learning rate 5×10^{-4}, batch size 64, discount factor $\gamma = 0.95$, a gradient update every 5 environment steps, gradient-norm clipping at 10, an ε-greedy training policy decaying linearly from 1.0 to 0.05 over the first 100,000 environment steps, and a fully greedy evaluation policy.

Within each training run, every episode samples one level $L \in \mathcal{D}_{\text{train}}$ uniformly with replacement; the episode horizon equals the generator's $T_{\max}$. Every 10,000 environment steps we run a greedy evaluation of 50 episodes on $\mathcal{D}_{\text{train}}$ and 50 episodes on $\mathcal{D}_{\text{test}}$, sampling levels uniformly from each pool. After the final training step we run a longer evaluation of 200 episodes per pool; the success rate $\hat{s}$ reported below is this final greedy estimate, with one estimate per (algorithm, seed) cell:

$$\hat{s}_{\text{train}}(a, s) = \frac{1}{200} \sum_{i=1}^{200} \mathbb{1}[\text{episode } i \text{ reaches all exits}]$$

and analogously for $\hat{s}_{\text{test}}(a, s)$.

The headline aggregate per algorithm $a \in \mathcal{A}$ is the mean and standard deviation over the twenty seeds; we also report a 95 % confidence interval $\pm 1.96 \frac{\sigma}{\sqrt{20}}$ in the per-seed appendix tables.

7.9.3 Results — Phase 1

Figure 10 shows the learning curves and Figure 11 the final greedy success rates. All three algorithms converge to $\hat{s} = 1.000$ on both $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$ for every seed (60 / 60 runs at 100 % final success; per-seed values in Section D.14). The standard deviation across seeds is exactly zero, so the per-algorithm confidence interval is degenerate. The three algorithms differ only in the time at which the greedy success rate first plateaus: VDN reaches 1.0 earliest, at approximately 5.5×10^4 environment steps, followed by IQL at approximately 7.0×10^4, and QMIX at approximately 8.0×10^4.

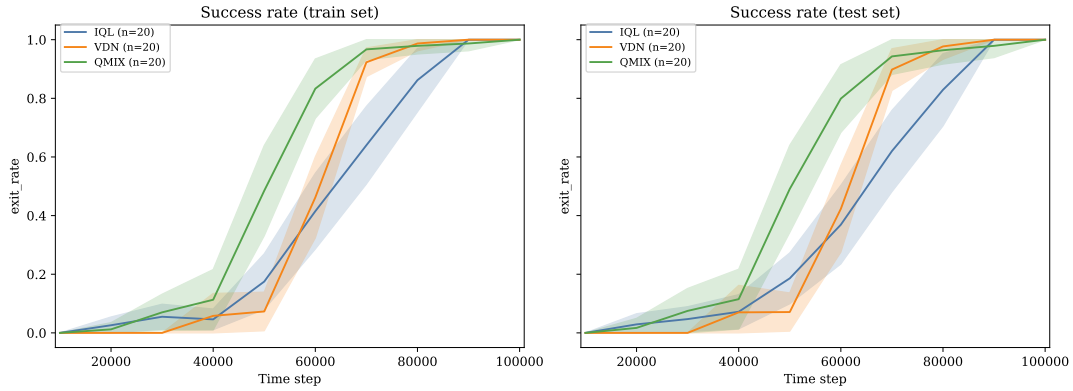

Figure 10: Phase 1 learning curves: greedy success rate on the train pool (solid) and test pool (dashed) as a function of environment steps, averaged over 20 seeds per algorithm; shaded bands are ± 1 standard deviation across seeds.

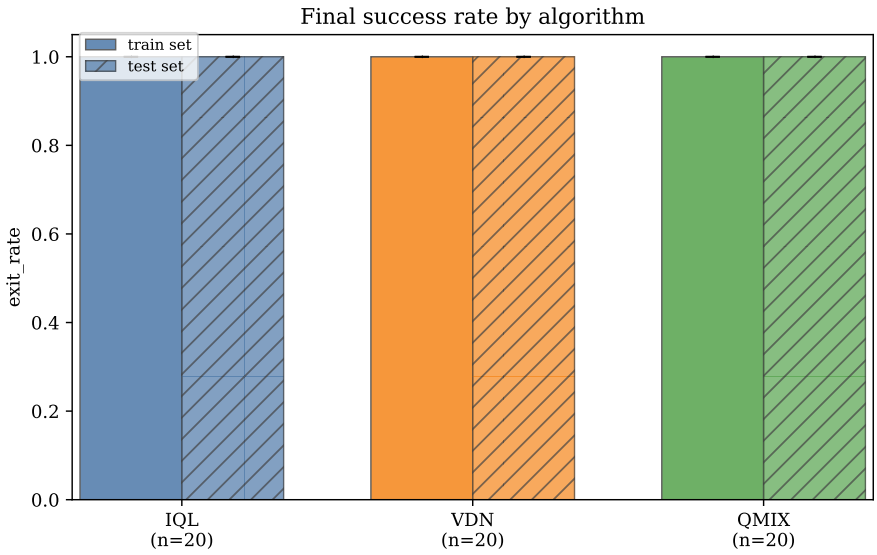

Figure 11: Phase 1 final greedy success rate after 100,000 environment steps, evaluated over 200 episodes per pool, per algorithm. All three algorithms reach 1.000 ± 0.000 on both pools.

The train / test gap on this configuration is identically zero. We read this as evidence that the 20-level cooperative pool on a 6×6 grid is small enough that any policy able to clear it generalises to the held-out pool of the same distribution; phase 1 does not discriminate between algorithms and does not exercise the cooperative generator's representational diversity.

7.9.4 Results — Phase 2

Figure 12 and Figure 13 report the same plots for the harder Phase 2 configuration. The per-algorithm final mean and standard deviation (over seeds 0–19, with QMIX additionally including seed 42 as a re-run of a Phase-2 partial; see footnote on Section D.15) are summarised in Table 3.

Algorithm	$\hat{s}_{\text{train}}$ (mean ± std)	$\hat{s}_{\text{test}}$ (mean ± std)
QMIX	0.85 ± 0.35	0.85 ± 0.35
VDN	0.60 ± 0.44	0.61 ± 0.45
IQL	0.16 ± 0.31	0.16 ± 0.30

Table 3: Phase 2 final greedy success rate after 200,000 environment steps, evaluated over 200 episodes per pool. Mean and standard deviation across $n = 20$ training seeds (QMIX includes one re-run, $n = 21$). Aggregates computed from the raw per-seed file `results/learnability_phase2/runs/{algo}_seed{N}/final_results.json`.

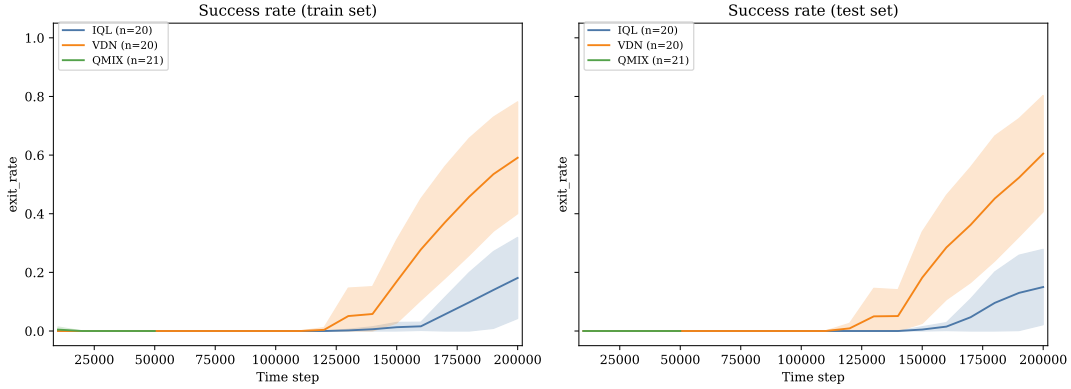

Figure 12: Phase 2 learning curves: greedy success rate on the train pool (solid) and test pool (dashed) as a function of environment steps, averaged over training seeds; shaded bands are ± 1 standard deviation across seeds.

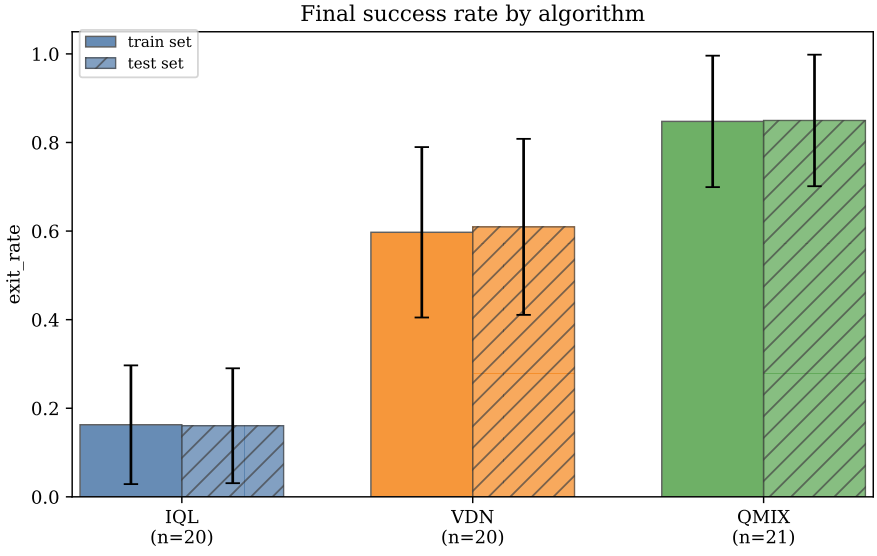

Figure 13: Phase 2 final greedy success rate after 200,000 environment steps, evaluated over 200 episodes per pool, per algorithm. Error bars are ± 1 standard deviation across seeds.

Two observations follow. First, the train / test gap is again essentially zero across all three algorithms ($|\hat{s}_{\text{train}} - \hat{s}_{\text{test}}| \leq 0.02$ for every algorithm in the phase-2 mean), which is the most direct evidence we have that the cooperative generator does not over-fit to a particular training pool: a policy that solves the 20 training levels solves the 20 unseen test levels at the same rate. Second, the variance across seeds

is large: for IQL and VDN the standard deviation across seeds exceeds the mean itself, reflecting the bimodal nature of the per-seed distribution. Inspecting the per-seed table (Section D.15), most seeds either converge fully ($\hat{s} \geq 0.95$) or fail to learn at all ($\hat{s} \leq 0.05$); only a small fraction sit at intermediate values. This bimodality concentrates the difference between algorithms in the **fraction of seeds that converge**, rather than in the asymptotic success rate of a converged seed.

7.9.5 Discussion

Phase 1 is undiscriminating: all three algorithms saturate the 6×6 cooperative pool quickly enough that the differences between IQL, VDN and QMIX are reduced to a small advantage in convergence time. Phase 2 separates the algorithms by credit-assignment strength, in the expected order: QMIX > VDN > IQL.

The most informative diagnostic is the train / test gap, not the final mean. The cooperative generator is the only direct dependency of the training distribution in this experiment, and the absence of a measurable train / test gap in either phase indicates that the 20 cooperative levels in the training pool already span the cooperative-level distribution well enough that an unseen 20-level sample from the same generator does not require additional adaptation. This is a necessary condition for the curriculum-transfer experiment in Section 7.10, which assumes that performance on a held-out generator pool is a valid proxy for the underlying cooperative distribution.

7.10 Curriculum Transfer to Level-6-Style Targets

7.10.1 Experimental Question

The learnability experiment above demonstrates that off-the-shelf cooperative MARL can solve levels drawn from the `cooperative` generator at 8×8 with three agents (Phase 2). The canonical hand-crafted target of this thesis — LLE Level 6 — is qualitatively harder: a 12×13 corridor-like map with four agents and three lasers in which all agents must pass through a shared corridor. Direct training on a single level of that size and structure is known to be brittle for value-decomposition methods.

We therefore ask whether **staged exposure** through a four-stage curriculum of generated levels transfers better to the Level-6-style target than three control conditions that skip the curriculum. The generators in the curriculum are the same ones characterised in Section 7.6 and Section 7.7, used here as a controlled source of training instances rather than as standalone artefacts.

7.10.2 Curriculum Stages

The curriculum is defined by the four `StageConfig` entries of `src/experiments/curriculum/configs.py`. Every stage uses four agents (so that all stages share the same observation and action spaces, and the trained Q-networks remain compatible across stage transitions) but varies grid size, laser count, horizon, and generator. The stages are summarised in Table 4 and rendered as sample grids in Section D.10–Section D.13.

Stage	Grid	Lasers	$T_{\max}$	Generator (thesis name)	Pool size
1	6×6	0	12	`constrained_random_solvable`	50 train
2	8×8	1	16	`constructive_cooperative`	50 train
3	10×10	2	18	`constructive_cooperative`	50 train
4	12×13	3	21	`constructive_level6_style`	50 train + 50 eval

Table 4: Curriculum stages used by the CURR condition. Generator names are the descriptive thesis-side names from `THESIS_GENERATOR_NAMES`; the corresponding runtime registry keys are `random`, `cooperative`, `cooperative`, `level6_style`. All stages use 4 agents.

Stages 1–3 each provide a 50-level training pool; stage 4 additionally provides a 50-level held-out evaluation pool that is never seen during training. Pool seeds are derived deterministically from `RNG_SEED = 20260514` and are documented in the per-pool `params.json` files under `results/curriculum_experiment/levels/`.

7.10.3 Conditions

We compare four conditions, all evaluated on the same hand-crafted Level 6 target. Each condition uses a single trainer (shared across stages where applicable), so the Q-network, replay buffer, and optimiser persist between stage transitions; only the source of training levels and the active $T_{\max}$ change.

- **B1 (target-only baseline).** Train exclusively on the stage-4 training pool (50 level-6-style generated levels). This isolates the contribution of the level-6-style generator without any curriculum.
- **B2 (uniform-mix baseline).** Train on the union of all four stage training pools (200 levels total), sampling uniformly from the union at every episode. This isolates the contribution of level **diversity** without the staged ordering of CURR.
- **B3 (oracle baseline).** Train exclusively on the hand-crafted Level 6, sampling that single level at every episode. This is the "no procedural generation at all" control.
- **CURR (curriculum).** Train on the four stages in order, advancing from stage k to stage $k+1$ as soon as the rolling success rate over the last `ADVANCEMENT_WINDOW_EPISODES = 100` training episodes reaches `ADVANCEMENT_SUCCESS_THRESHOLD = 0.80`. If the threshold is not met within the per-stage step cap (`per_stage_step_cap_*`, 375,000 steps per stage for the full budget; halved for the pilot), the scheduler advances anyway to prevent lock-up on a hard stage.

The four conditions share the same total training budget. For the full experiment that budget is `FULL_RUN_TOTAL_STEPS = 1,500,000` environment steps; for the pilot reported here it is `PILOT_RUN_TOTAL_STEPS = 750,000` environment steps.

7.10.4 Evaluation

The headline metric is the greedy success rate on the hand-crafted Level 6, denoted $\hat{s}_{\text{level6}}$, estimated from `FINAL_EVAL_EPISODES = 200` independent episodes after the last training step (epsilon = 0). Periodic evaluations of `EVAL_EPISODES = 50` episodes are logged every `EVAL_FREQUENCY_STEPS = 20,000` environment steps in `level6_eval.csv`. For the B1 condition we additionally report the greedy success rate on the 50-level stage-4 held-out pool (`success_rate_held_out_pool` in `final_results.json`), which probes whether B1 over-fits to its own 50-level training pool.

A positive transfer result is: $\hat{s}_{\text{level6}}(\text{CURR}) > \hat{s}_{\text{level6}}(\text{B1})$ and $> \hat{s}_{\text{level6}}(\text{B2})$ at the same total step budget. A weaker but still informative result is: CURR reaches its asymptote in fewer environment steps than the baselines.

7.10.5 Pilot Setup

At the time of writing, only the **pilot** run is in progress: two seeds of QMIX per condition ($|\mathcal{S}_{\text{pilot}}| = 2$, $|\mathcal{A}_{\text{pilot}}| = \{\text{QMIX}\}$) at the `PILOT_RUN_TOTAL_STEPS = 750,000` budget. The pilot's purpose is to verify that the curriculum scheduler, the per-stage advancement criterion, and the eval cadence behave as specified before committing the full (1.5×10^6 steps) $\times$ (3 algos) $\times$ (multiple seeds) run. The full design space is the cartesian product

$$\mathcal{C} \times \mathcal{A} \times \mathcal{S} = \{\text{B1}, \text{B2}, \text{B3}, \text{CURR}\} \times \{\text{QMIX}, \text{VDN}, \text{IQL}\} \times \mathcal{S}$$

which we will populate once the pilot's stability is confirmed.

7.10.6 Results

Results of the QMIX pilot will be reported once all four condition cells have completed their 750,000-step budget; see the placeholder above for the figures and table to insert.

7.10.7 Anticipated Interpretation

If CURR outperforms B1 at the pilot budget, the staged exposure provides curricular value beyond what level-6-style training alone supplies, and the procedural-generator framework of Section 6 is validated as a curriculum source for cooperative MARL on the Level-6 target. If CURR matches B2 (uniform mix) but neither outperforms B3 (single Level 6), the bottleneck is sample efficiency of QMIX on this target rather than the training distribution. If CURR underperforms B1, the implicit smoothness assumption of the curriculum — that policies useful at stage k remain useful at stage $k+1$ — is violated for this specific generator sequence, motivating a redesign of the stage geometry rather than a different curriculum scheduler.

Conclusion

8.1 Summary

This thesis developed a SAT-based framework for reasoning about solvability and cooperation in a modeled subset of the Laser Learning Environment. We first formalised bounded-horizon solvability as a decision problem, then reduced it to satisfiability of a CNF formula. On top of that reduction, we introduced a strict beam-semantics counterfactual that turns the blocking-based cooperation mechanism of LLE into a second decision problem on the same level.

These decision procedures were then embedded inside a family of procedural generators. The resulting framework does not guarantee that generated levels are pedagogically optimal for MARL, but it does guarantee that accepted levels satisfy the formal properties checked by the solver. This is the central contribution of the thesis: procedural generation coupled to explicit certification rather than procedural generation guided only by heuristic plausibility.

The empirical evaluation covers several aspects of the framework. The SAT encoding comparison shows that the local uniqueness formulation is the preferable default: it yields substantially smaller CNF formulas and lower runtimes as instances grow. The rejection-rate and profile-distribution experiments characterise the practical cost of the acceptance oracle and the output diversity of the generator family. A transfer experiment is planned to evaluate whether agents trained on certified cooperative levels can generalise to human-designed benchmark levels.

8.2 Future Work

Several directions extend the present work naturally.

- **Parameter sensitivity and diversity.** The current acceptance-rate experiments use fixed agent and laser counts per grid size. A fuller characterisation would vary these parameters to locate the practical frontier at which rejection rates become prohibitive, and would measure diversity of accepted levels within a fixed parameter setting.
- **Richer cooperation metrics.** The cooperation profile taxonomy used here covers five dependency structures. Extensions could include synchronous cooperation (agents that must coordinate in the same timestep) and chain length as a difficulty axis. These richer targets are already defined in the cooperation profile notes; their generation and evaluation remain future work.
- **Model extension.** Incorporating gem tiles and void tiles into the formal model would expand the scope of the formal guarantees. Gems introduce an additional reachability condition; void tiles require a revised movement semantics. Both extensions preserve the SAT-based architecture.

- **Formal NP-hardness.** The present thesis shows that bounded-horizon LLE solvability is in NP and is reducible to SAT. Whether it is NP-hard — that is, whether there exists a polynomial-time reduction from a known NP-hard problem to LLE solvability — remains open and would be a worthwhile theoretical complement.
- **Curriculum learning over certified levels.** Because the acceptance oracle controls difficulty through generation parameters (grid size, agent and laser counts, horizon, target cooperation profile), the generator family naturally supports a **curriculum** [24], [25]: agents could be trained first on small, easy certified levels with simple cooperation patterns, then exposed to progressively larger grids and richer profiles. This matches the curriculum-learning hypothesis that a graded sequence of training tasks helps agents acquire structured behaviours that direct training on the hardest instances alone would fail to reach. The fact that every level in such a curriculum is solver-certified guarantees the cooperation signal is present at every stage of training.

The main open question is therefore not whether solver-based certification is possible in LLE; the present thesis answers that positively. The open question is how far that certification framework can be extended — in terms of richer mechanics, richer cooperation structures, and downstream learning effects — before additional formal layers are required.

Appendix

D.3 Training Level Set

This appendix lists the levels used to train agents in the transfer experiment (Section 7.10). All levels were generated by the cooperative generator framework and certified by the SAT-based acceptance oracle. For each level, the grid size, agent count, laser count, and cooperation profile are reported.

To be completed: insert level images and parameter table here.

D.4 Benchmark Levels for SAT Encoding Comparison

The four levels used in the SAT encoding comparison (Section 7) are shown below with their exact parameters.

Level	Grid	Agents	Lasers	Horizon $T_{\max}$
Synthetic 3×3	3×3	2	1	8
Synthetic 5×5	5×5	3	2	12
Synthetic 8×8	8×8	4	3	32
Benchmark Level 6	varies	4	3	known

Table 5: Parameters of the four benchmark levels used in the SAT encoding comparison.

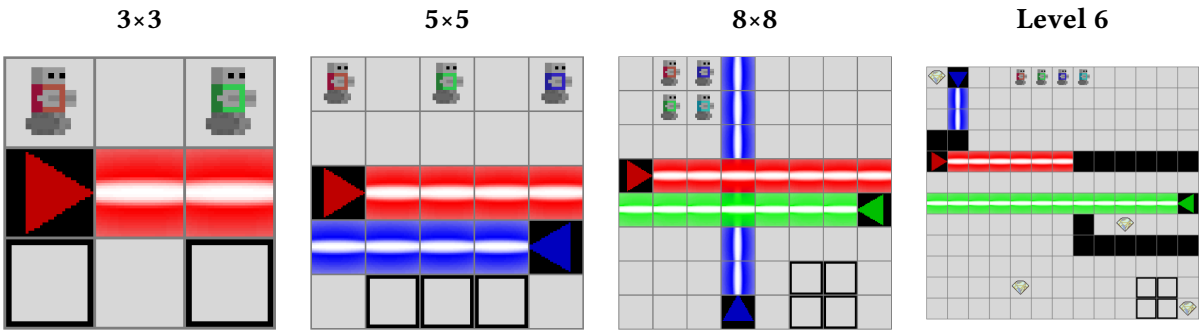

Figure 14: Visual representation of the four benchmark levels.

D.5 Generator Parameters

Parameter	3×3	5×5	8×8
Agents	2	3	4
Lasers	1	2	3
$T_{\max}$ (rejection benchmark)	8	12	20
$T_{\max}$ (profile benchmark)	—	12	14
Trials (rejection benchmark)	200	200	20
Max attempts per trial (rejection benchmark)	500	500	100
Trial timeout, s (rejection benchmark, 8×8 only)	—	—	30
Levels generated (profile benchmark)	—	100	50

Table 6: Generator parameters used in the rejection rate and profile distribution benchmarks.

D.6 Cooperation Profile Examples

The cooperation profile analyzer classifies accepted cooperative levels into five structural families based on the inter-agent dependency graph. The definitions are given in Section 6. Representative visual examples of each profile type are provided below once the generator benchmarks are completed.

- **Asymmetric**: Agent A depends on Agent B, but not vice versa.
- **Mutual**: Agents A and B each depend on the other.
- **Chain**: Dependencies form a directed chain A $\longrightarrow$ B $\longrightarrow$ C.
- **Distributed**: One agent depends on multiple distinct helpers.
- **Fully coupled**: All agents belong to a single strongly connected dependency component.

To be completed: insert representative level images for each profile type.

D.7 Learnability and Curriculum Hyperparameters

The learnability experiment (Section 7.9) and the curriculum-transfer experiment (Section 7.10) share the same trainer construction in `src/experiments/learnability/run_experiment.py` and `src/experiments/curriculum/run_experiment.py`. The hyperparameters listed in Table 7 are identical across the three algorithms (IQL, VDN, QMIX) and across both phases of the learnability experiment, as well as across the four conditions (B1, B2, B3, CURR) of the curriculum-transfer experiment.

Hyperparameter	Value	Source
Optimiser	Adam	`marl.algos.{DQN,VDN,QMix}` default
Learning rate	5×10^{-4}	`run_experiment.py` (`lr=5e-4`)
Batch size	64	`run_experiment.py` (`batch_size=64`)
Discount factor γ	0.95	`run_experiment.py` (`gamma=0.95`)
Train interval	1 update / 5 env steps	`run_experiment.py` (`train_interval=(5,"step`
Gradient-norm clipping	10.0	`run_experiment.py` (`grad_norm_clipping=10`)
ε schedule (train)	linear $1.0 \to 0.05$ over 100,000 steps	`EpsilonGreedy.linear(1.0, 0.05, 100_000)`
Evaluation policy	greedy ($\varepsilon = 0$)	`ArgMax()`
Q-network architecture	marl `qnetworks.from_env` default	TODO: confirm exact RNN topology in marl
Mixer (QMIX)	`mixers.QMix.from_env`	`run_experiment.py`
Mixer (VDN)	sum of agent Q-values	`marl.algos.VDN` implicit mixer
Mixer (IQL)	none	`run_experiment.py` (`mixer=None`)
Independent Q-network heads (IQL, VDN)	yes	`qnetworks.from_env(..., independent=True)`
Independent Q-network heads (QMIX)	no (shared)	`qnetworks.from_env(...)` default
Replay buffer size	marl default	TODO: confirm in marl source
Target network update interval	marl default	TODO: confirm in marl source

Table 7: Hyperparameters used in both the learnability experiment (Section 7.9) and the curriculum-transfer experiment (Section 7.10). Rows marked as marl defaults inherit from the upstream marl framework (`C:/Users/hugoc/Projects/marl`); the commit pinned for these runs is recorded in `results/learnability/runs/marl_commit.txt`.

D.8 Learnability Pool Samples — Phase 1

The Phase 1 cooperative pool is a 6×6 grid with 2 agents and 1 laser, horizon $T_{\max} = 10$, generated by the `cooperative` registry generator. Pool-generation parameters are recorded in `results/learnability/levels/6x6_2a_1L_cooperative/params.json` (train seed 20260515, test seed 20260516, 20 levels per split).

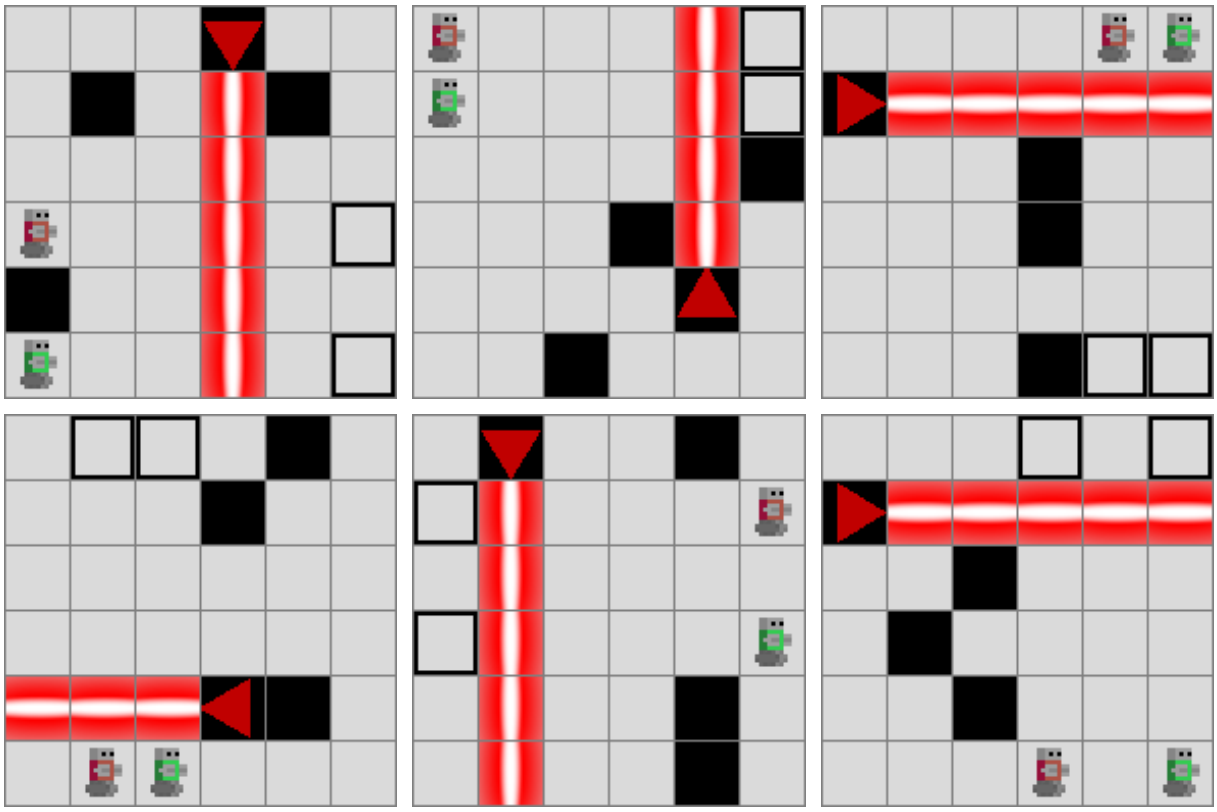

Figure 15: Six representative training levels from the Phase 1 cooperative pool (6×6, 2 agents, 1 laser, $T_{\max} = 10$, generator `cooperative`).

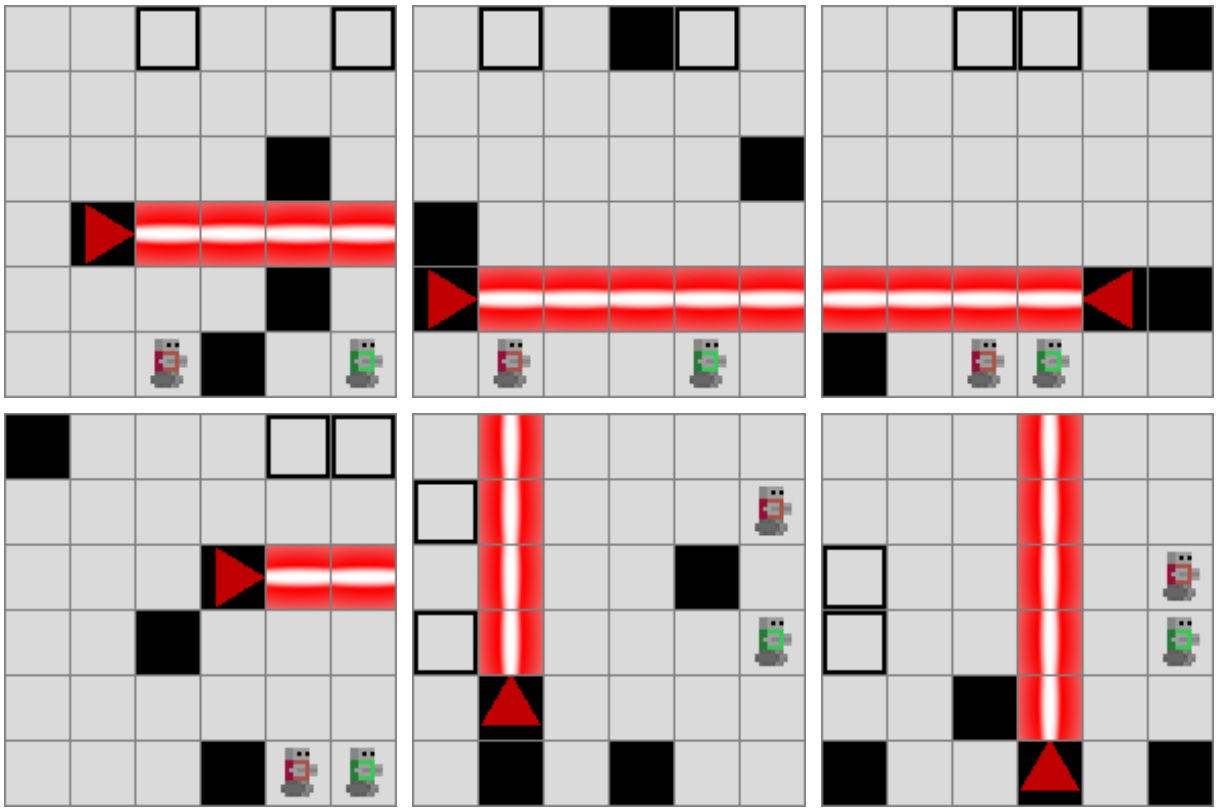

Figure 16: Six representative held-out test levels from the Phase 1 cooperative pool (6×6, 2 agents, 1 laser, $T_{\max} = 10$, generator `cooperative`).

D.9 Learnability Pool Samples — Phase 2

The Phase 2 cooperative pool is an 8×8 grid with 3 agents and 2 lasers, horizon $T_{\max} = 16$, generated by the `cooperative` registry generator. Pool-generation parameters are recorded in `results/learnability_phase2/levels/8x8_3a_2L_cooperative/params.json`.

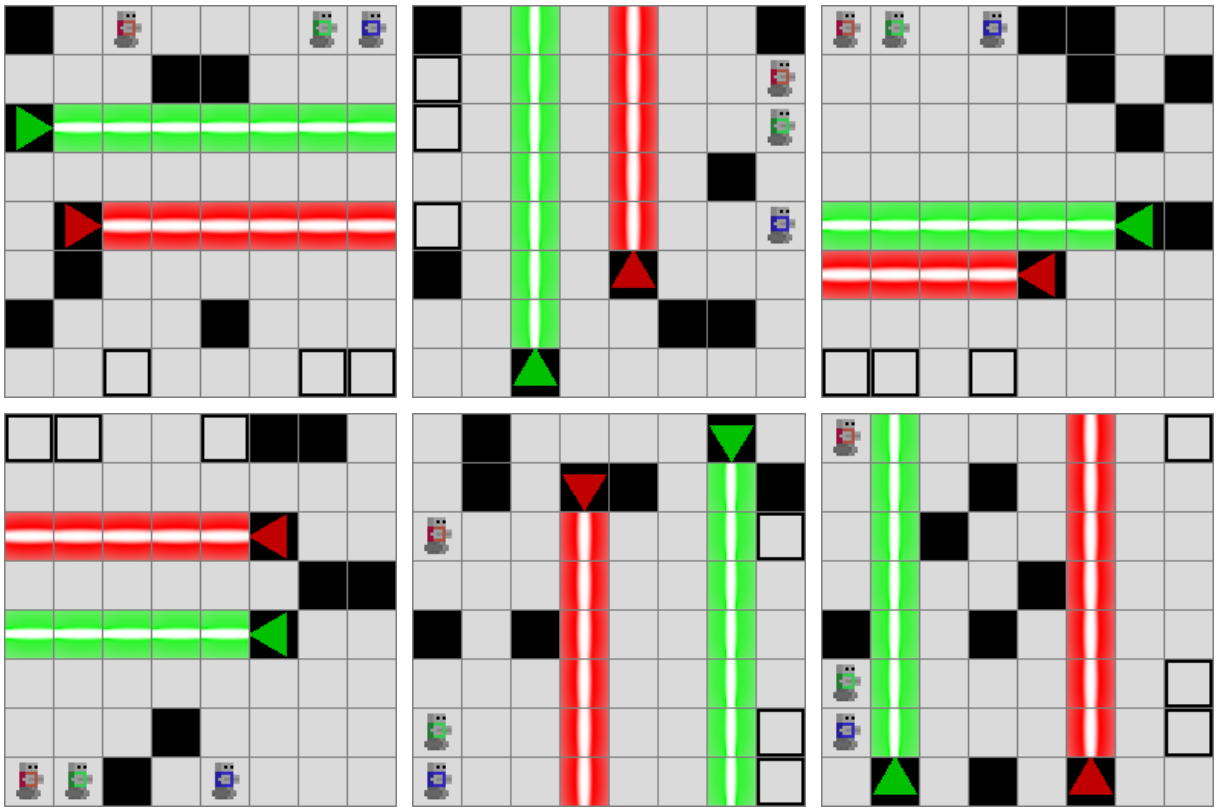

Figure 17: Six representative training levels from the Phase 2 cooperative pool (8×8, 3 agents, 2 lasers, $T_{\max} = 16$, generator `cooperative`).

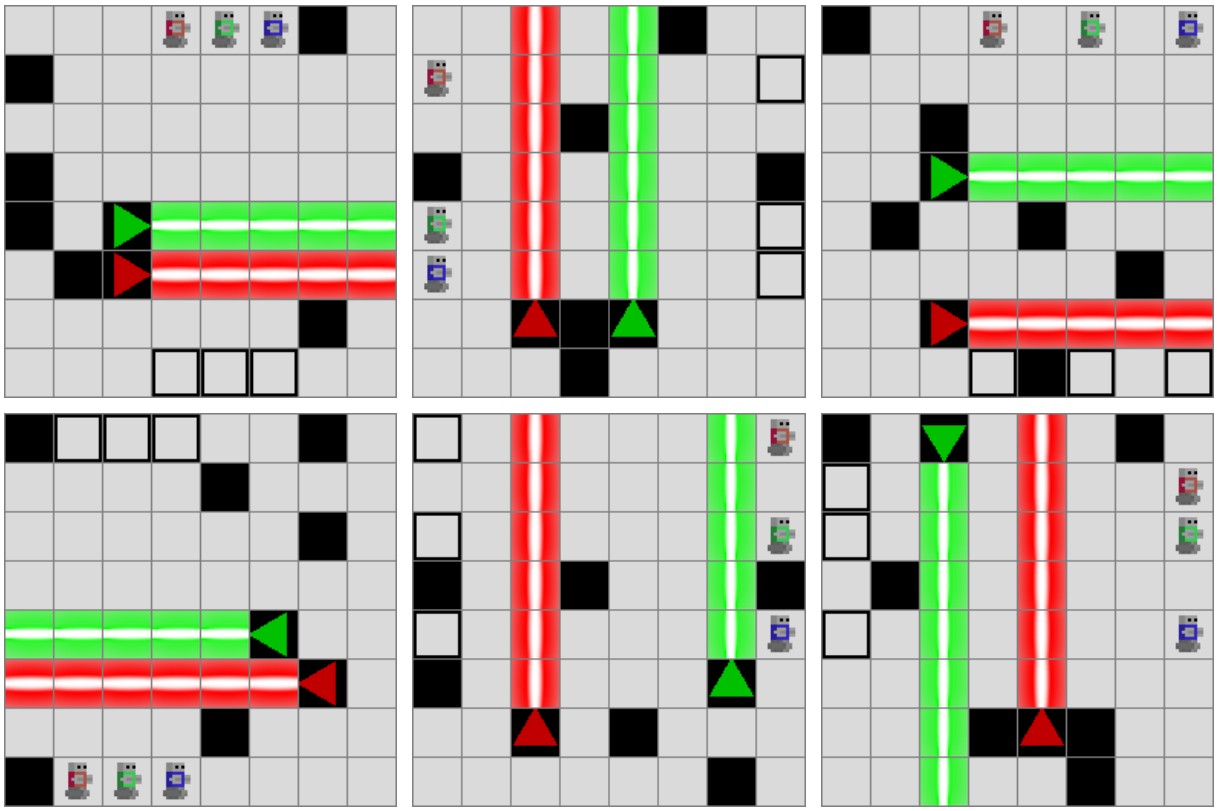

Figure 18: Six representative held-out test levels from the Phase 2 cooperative pool (8×8, 3 agents, 2 lasers, $T_{\max} = 16$, generator `cooperative`).

D.10 Curriculum Pool Samples — Stage 1

Stage 1 of the curriculum is a 6×6 grid with 4 agents and 0 lasers, horizon $T_{\max} = 12$, generated by the `random` registry generator (`constrained_random_solvable` in the thesis names). With 0 lasers the cooperation requirement is structurally absent: stage 1 is a pure-navigation warmup that exposes agents to the 4-agent action space before the cooperative stages introduce same-colour beam-truncation. Pool seed: 20260614.

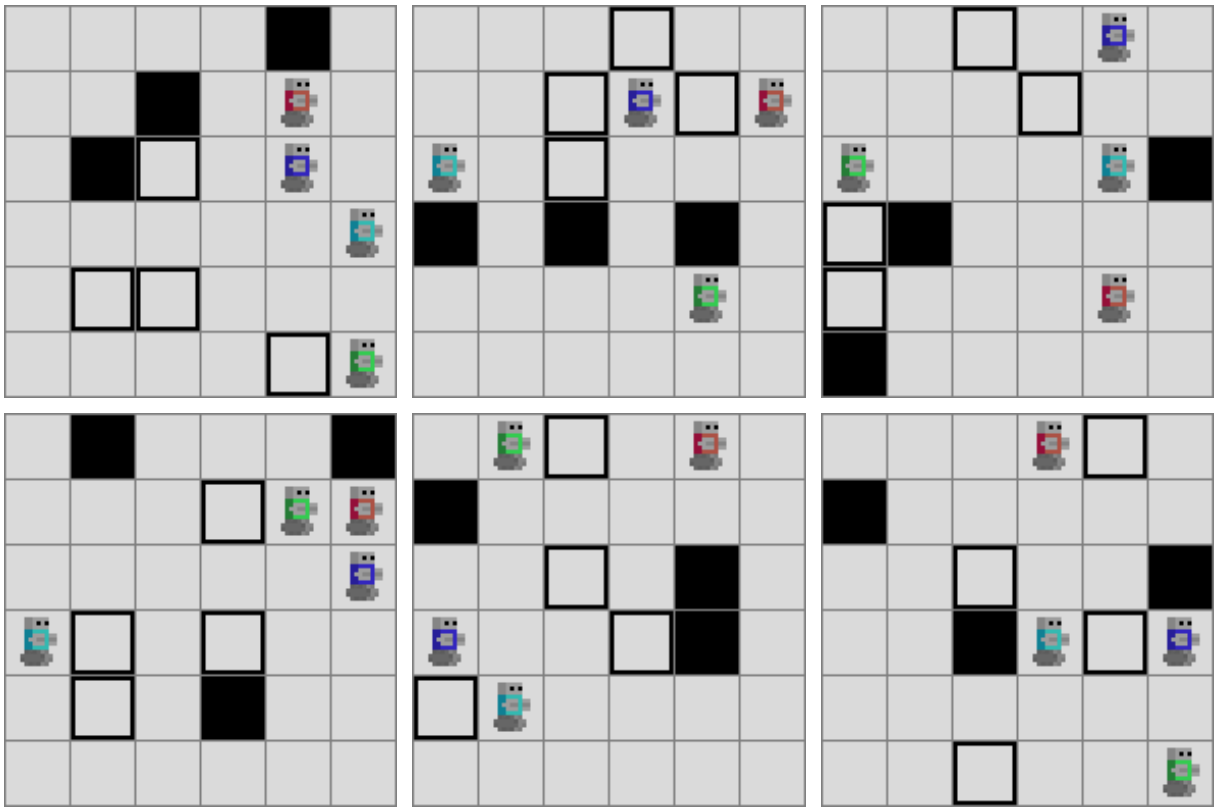

Figure 19: Six representative training levels from curriculum stage 1 (6×6, 4 agents, 0 lasers, $T_{\max} = 12$, generator `random`).

D.11 Curriculum Pool Samples — Stage 2

Stage 2 of the curriculum is an 8×8 grid with 4 agents and 1 laser, horizon $T_{\max} = 16$, generated by the `cooperative` registry generator (`constructive_cooperative` in the thesis names). Pool seed: 20260714.

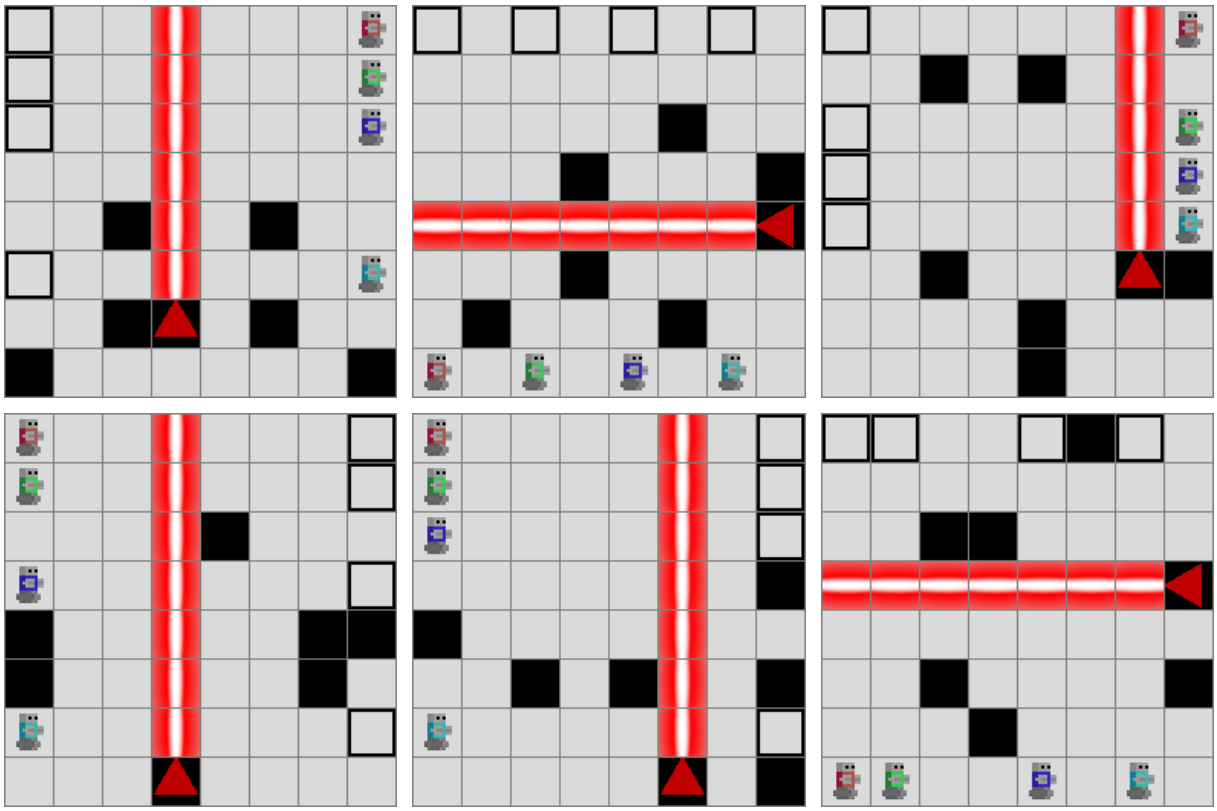

Figure 20: Six representative training levels from curriculum stage 2 (8×8, 4 agents, 1 laser, $T_{\max} = 16$, generator `cooperative`).

D.12 Curriculum Pool Samples — Stage 3

Stage 3 of the curriculum is a 10×10 grid with 4 agents and 2 lasers, horizon $T_{\max} = 18$, generated by the `cooperative` registry generator. Pool seed: 20260814.

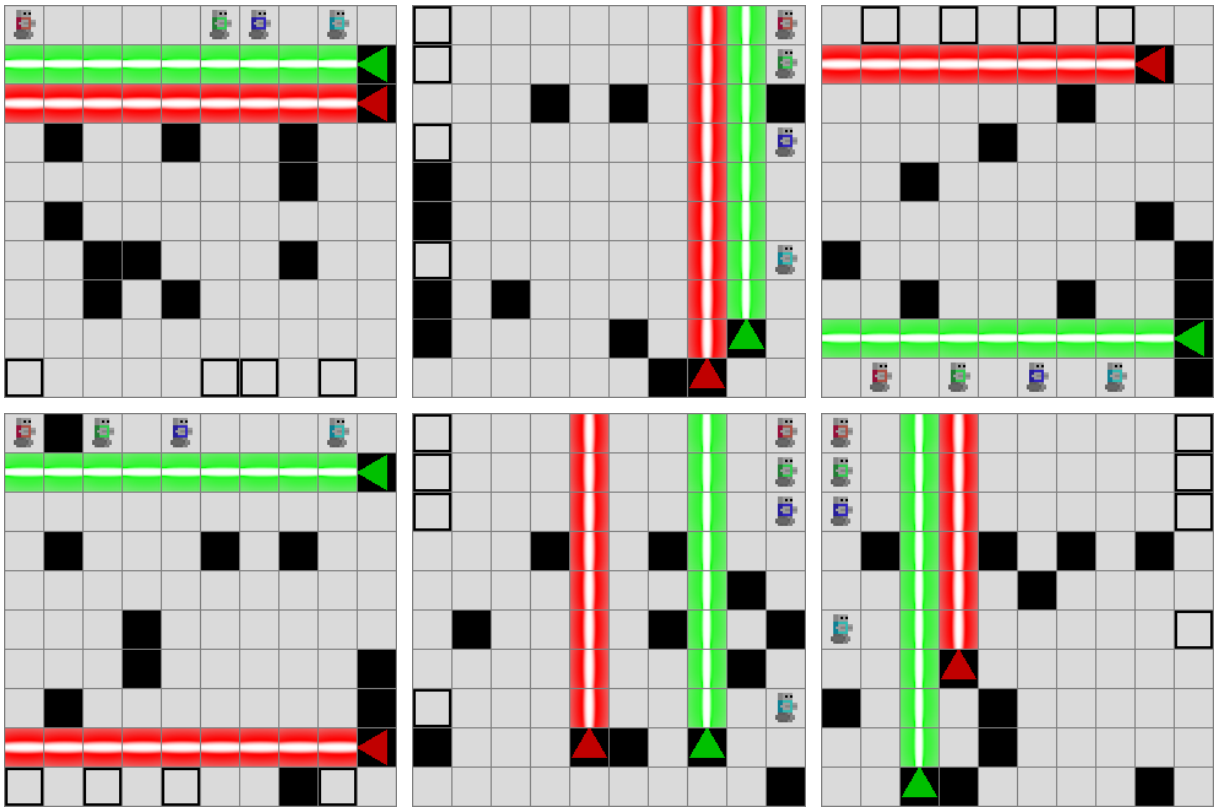

Figure 21: Six representative training levels from curriculum stage 3 (10×10, 4 agents, 2 lasers, $T_{\max} = 18$, generator `cooperative`).

D.13 Curriculum Pool Samples — Stage 4

Stage 4 of the curriculum is a 12×13 grid with 4 agents and 3 lasers, horizon $T_{\max} = 21$, generated by the `level6_style` registry generator (`constructive_level6_style` in the thesis names). Pool seeds: 20260914 (train), 20260915 (eval). This stage is also the **transfer target** of the B1 baseline of Section 7.10.

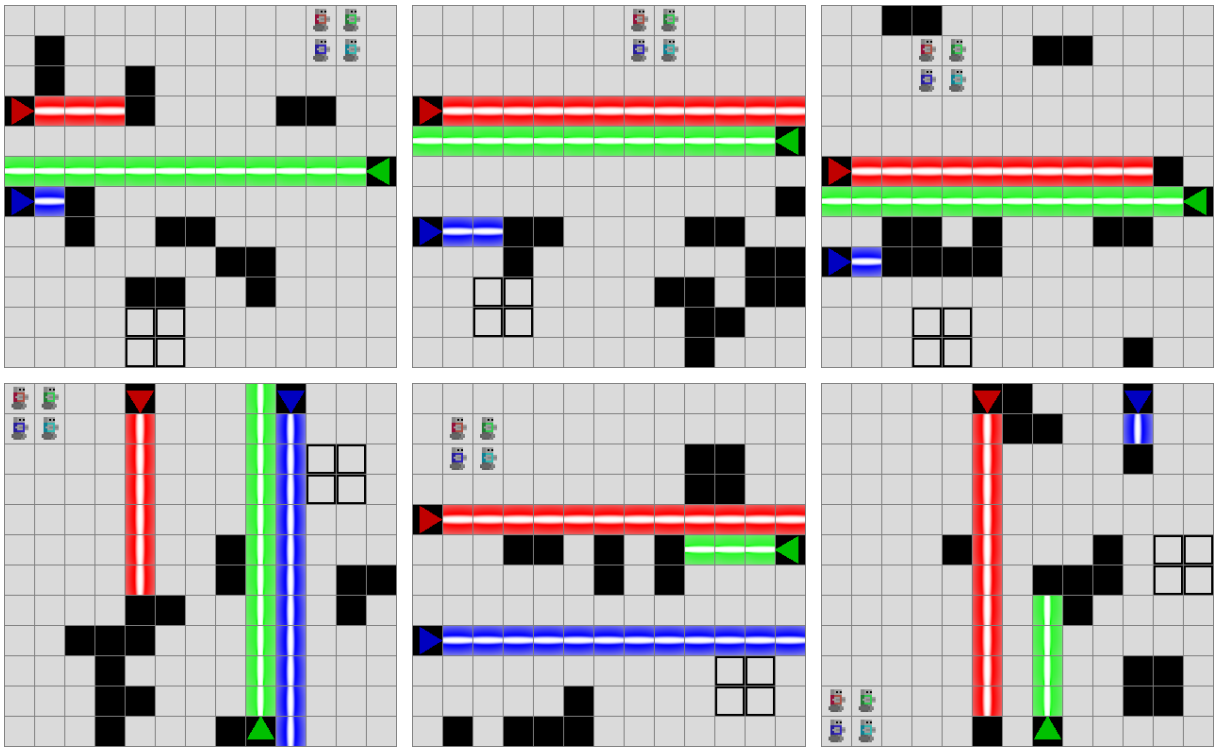

Figure 22: Six representative training levels from curriculum stage 4 (12×13, 4 agents, 3 lasers, $T_{\max} = 21$, generator `level6_style`).

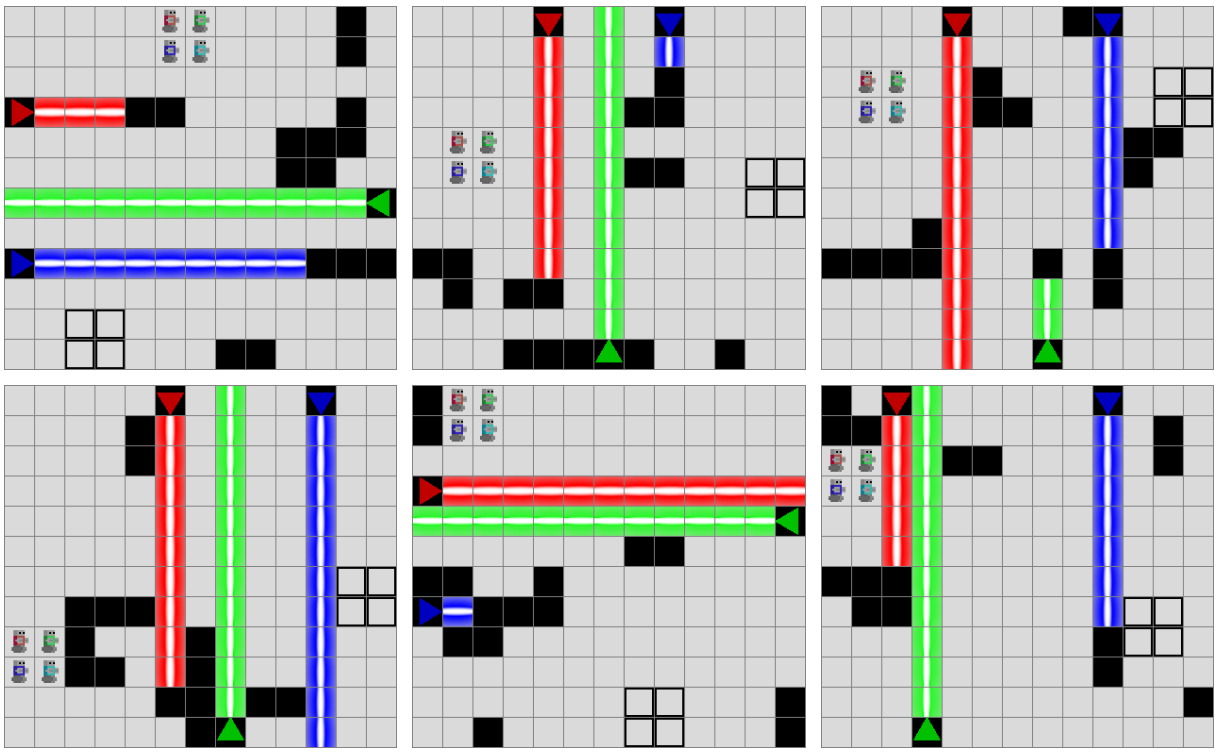

Figure 23: Six representative held-out evaluation levels from curriculum stage 4 (12×13, 4 agents, 3 lasers, $T_{\max} = 21$, generator `level6_style`). Used as the held-out generated test set for the B1 baseline of Section 7.10.

D.14 Per-Seed Final Success Rates — Phase 1

Per-seed final greedy success rates after 100,000 environment steps, evaluated over 200 episodes per pool. Source: `results/learnability/runs/{algo}_seed{N}/final_results.json`. All 60 runs (3 algorithms $\times$ 20 seeds) reach $\hat{s} = 1.000$ on both pools, so the mean is 1.000 and the standard deviation is 0.000 for every algorithm.

Seed	IQL train	IQL test	VDN train	VDN test	QMIX train	QMIX test
0	1.000	1.000	1.000	1.000	1.000	1.000
1	1.000	1.000	1.000	1.000	1.000	1.000
2	1.000	1.000	1.000	1.000	1.000	1.000
3	1.000	1.000	1.000	1.000	1.000	1.000
4	1.000	1.000	1.000	1.000	1.000	1.000
5	1.000	1.000	1.000	1.000	1.000	1.000
6	1.000	1.000	1.000	1.000	1.000	1.000
7	1.000	1.000	1.000	1.000	1.000	1.000
8	1.000	1.000	1.000	1.000	1.000	1.000
9	1.000	1.000	1.000	1.000	1.000	1.000
10	1.000	1.000	1.000	1.000	1.000	1.000
11	1.000	1.000	1.000	1.000	1.000	1.000
12	1.000	1.000	1.000	1.000	1.000	1.000
13	1.000	1.000	1.000	1.000	1.000	1.000
14	1.000	1.000	1.000	1.000	1.000	1.000
15	1.000	1.000	1.000	1.000	1.000	1.000
16	1.000	1.000	1.000	1.000	1.000	1.000
17	1.000	1.000	1.000	1.000	1.000	1.000
18	1.000	1.000	1.000	1.000	1.000	1.000
19	1.000	1.000	1.000	1.000	1.000	1.000
mean	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**
std	0.000	0.000	0.000	0.000	0.000	0.000

Table 8: Phase 1: per-seed final greedy success rate after 100,000 environment steps. Each cell is the fraction of 200 evaluation episodes in which all agents reached an exit. Mean and standard deviation are computed across the 20 seeds in the column.

D.15 Per-Seed Final Success Rates — Phase 2

Per-seed final greedy success rates after 200,000 environment steps, evaluated over 200 episodes per pool. Source: `results/learnability_phase2/runs/{algo}_seed{N}/final_results.json`. The mean and standard deviation rows are taken over all listed seeds for that algorithm (including the QMIX re-run; see footnote).

Seed	IQL train	IQL test	VDN train	VDN test	QMIX train	QMIX test
0*	0.000	0.000	0.000	0.000	0.000	0.000
1	0.000	0.000	1.000	0.955	1.000	0.985
2	0.000	0.000	0.955	1.000	1.000	1.000
3	0.000	0.000	0.000	0.000	1.000	1.000
4	0.000	0.000	0.965	1.000	0.000	0.000
5	0.535	0.445	1.000	1.000	0.965	0.965
6	0.130	0.130	0.700	0.960	0.000	0.000
7	0.000	0.000	0.860	1.000	1.000	1.000
8	0.000	0.000	0.000	0.000	1.000	1.000
9	0.070	0.210	1.000	0.930	1.000	1.000
10	0.110	0.060	0.940	1.000	1.000	0.965
11	0.000	0.000	0.930	0.760	1.000	1.000
12	0.000	0.000	0.085	0.000	0.905	0.960
13	1.000	0.970	0.640	0.705	0.930	1.000
14	0.040	0.065	0.000	0.000	1.000	1.000
15	0.000	0.000	0.910	1.000	1.000	1.000
16	0.255	0.210	0.000	0.000	1.000	1.000
17	0.115	0.120	1.000	0.950	1.000	1.000
18	0.000	0.000	0.960	0.930	1.000	1.000
19	1.000	1.000	0.000	0.000	1.000	1.000
42**	–	–	–	–	1.000	0.970
mean	**0.163**	**0.161**	**0.597**	**0.610**	**0.848**	**0.850**
std	0.306	0.296	0.439	0.453	0.347	0.347

Table 9: Phase 2: per-seed final greedy success rate after 200,000 environment steps. Each cell is the fraction of 200 evaluation episodes in which all agents reached an exit.

* Seed 0 of QMIX completed only 50,000 of the 200,000 step budget (partial run); the value is reported as-is and is included in the QMIX mean / std.

** Seed 42 of QMIX is an additional 200,000-step run added to compensate for the seed 0 partial. The IQL and VDN columns of this row are empty by design (only QMIX was re-run).

Bibliography

[1] L. S. Shapley, "Stochastic Games," *Proceedings of the National Academy of Sciences*, vol. 39, no. 10, pp. 1095–1100, 1953, doi: 10.1073/pnas.39.10.1095.

[2] M. L. Littman, "Markov Games as a Framework for Multi-Agent Reinforcement Learning," in *Machine Learning Proceedings 1994*, Morgan Kaufmann, 1994, pp. 157–163. doi: 10.1016/B978-1-55860-335-6.50027-1.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[4] P. Sunehag *et al.*, "Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, in AAMAS 2018. International Foundation for Autonomous Agents, Multi-agent Systems, 2018, pp. 2085–2087.

[5] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," in *Proceedings of the 35th International Conference on Machine Learning*, in ICML 2018, vol. 80. PMLR, 2018, pp. 4295–4304.

[6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Advances in Neural Information Processing Systems 30*, in NeurIPS 2017. 2017, pp. 6379–6390.

[7] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual Multi-Agent Policy Gradients," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, in AAAI-18. AAAI Press, 2018, pp. 2974–2982.

[8] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "MAVEN: Multi-Agent Variational Exploration," in *Advances in Neural Information Processing Systems 32*, in NeurIPS 2019. 2019, pp. 7613–7624.

[9] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, "Laser Learning Environment: A New Environment for Coordination-Critical Multi-Agent Tasks," in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[10] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016. doi: 10.1007/978-3-319-42716-4.

[11] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011, doi: 10.1109/TCIAIG.2011.2148116.

[12] A. M. Smith and M. Mateas, "Answer Set Programming for Procedural Content Generation: A Design Space Approach," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 187–200, 2011, doi: 10.1109/TCIAIG.2011.2158545.

[13] A. Summerville *et al.*, "Procedural Content Generation via Machine Learning (PCGML)," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018, doi: 10.1109/TG.2018.2846639.

[14] H. Kautz and B. Selman, "Planning as Satisfiability," in *Proceedings of the 10th European Conference on Artificial Intelligence*, in ECAI '92. John Wiley & Sons, 1992, pp. 359–363.

[15] H. Kautz and B. Selman, "Pushing the Envelope: Planning, Propositional Logic, and Stochastic Search," in *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, in AAAI-96. AAAI Press, 1996, pp. 1194–1201.

[16] J. Rintanen, K. Heljanko, and I. Niemelä, "Planning as satisfiability: parallel plans and algorithms for plan search," *Artificial Intelligence*, vol. 170, no. 12–13, pp. 1031–1080, 2006, doi: 10.1016/j.artint.2006.08.002.

[17] P. Surynek, "Problem Compilation for Multi-Agent Path Finding: A Survey," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, in IJCAI-22. 2022, pp. 5615–5620.

[18] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015, doi: 10.1016/j.artint.2014.11.006.

[19] M. Frommknecht and P. Surynek, "Solving Multi-Agent Pathfinding with Stochastic Local Search SAT Algorithms," in *Proceedings of the 21st International Conference on Informatics in Control, Automation and Robotics*, in ICINCO 2024, vol. 1. 2024, pp. 67–78. doi: 10.5220/0012944800003822.

[20] S. A. Cook, "The Complexity of Theorem-Proving Procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.

[21] M. Davis, G. Logemann, and D. Loveland, "A Machine Program for Theorem-Proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1962, doi: 10.1145/368273.368557.

[22] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing – SAT 2003*, in Lecture Notes in Computer Science, vol. 2919. Springer, 2004, pp. 502–518. doi: 10.1007/978-3-540-24605-3_37.

[23] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," in *Theory and Applications of Satisfiability Testing – SAT 2018*, in Lecture Notes in Computer Science, vol. 10929. Springer, 2018, pp. 428–437. doi: 10.1007/978-3-319-94144-8_26.

[24] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, in ICML 2009. ACM, 2009, pp. 41–48. doi: 10.1145/1553374.1553380.

[25] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.